Báo Cáo Phản Biện Đồ Án Kiến Trúc Hệ Thống Storix

TÀI LIỆU CHUẨN BỊ PHẢN BIỆN ĐỒ ÁN KIẾN TRÚC HỆ THỐNG STORIX

BỘ CÂU HỎI & CÂU TRẢ LỜI BẢO VỆ CHUYÊN SÂU TỪ CƠ BẢN ĐẾN NÂNG CAO

(Kèm sơ đồ kiến trúc Draw.io chi tiết và trích dẫn nguồn mã nguồn dự án / tiêu chuẩn kỹ thuật)

- **Dự án:** Storix – Enterprise Multi-Tenant Nextcloud SaaS on AWS
- **Đơn vị phát triển:** FilestoreCloud Architecture Team
- **Môi trường triển khai thực tế:** AWS Region `ap-southeast-1` (Singapore) | Account: `649519997247`
- **Tiêu chuẩn tuân thủ:** NIST SP 800-162 (ABAC/RBAC), RFC 4918 (WebDAV), FIDO2/WebAuthn, AWS Well-Architected Framework (SaaS Lens).

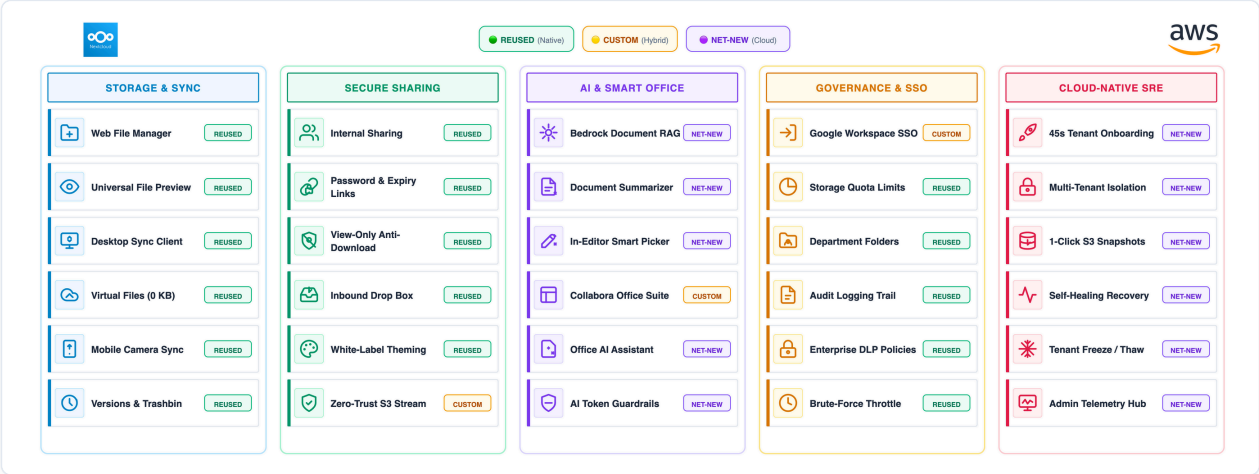
🧭 BẢN ĐỒ ĐIỀU HƯỚNG 12 CHỦ ĐỀ PHẢN BIỆN

STT	Chủ Đề Chuyên Môn	Số Lượng Câu	Trọng Tâm Phản Biện Của Giám Khảo
01	Bài Toán Nghiệp Vụ & Định Vị Dự Án	3 câu	Value Proposition, Zero-Code-Modification, RBAC 4 Actors
02	Kiến Trúc Mạng & Thiết Kế Zero-NAT	3 câu	Zero-NAT ($0 Egress), Packet Flow, Multi-Domain Routing
03	Mô Hình Cô Lập Đa Người Thuê	3 câu	Siloed vs Pooled, Noisy Neighbor, Lifecycle Statechart
04	Tầng Compute & Tối Ưu Container Runtime	4 câu	Graviton ARM64, AWS SOCI Cold Start, CAN_INSTALL, Auto-Scaling
05	Tầng Lưu Trữ Đa Cấp & Cơ Sở Dữ Liệu	5 câu	EFS vs S3, POSIX Access Point, RDS Proxy, S3/DB Consistency
06	Control Plane, SAGA Pattern & Bất Đồng Bộ	4 câu	Step Functions SAGA, Rollback bù trừ, DynamoDB OCC, Async OCC
07	Trí Tuệ Nhân Tạo GenAI & OpenSearch	3 câu	LiteLLM Sidecar 127.0.0.1:4000, Zero Data Leakage, OpenSearch Patch
08	Bảo Mật, Sinh Trắc Học & Phân Quyền	3 câu	WebAuthn Passkeys, WebDAV Exclusive Lock (HTTP 423), File Drop
09	FinOps & Kinh Tế Học Điện Toán	2 câu	Scale-to-Zero ($0.00 compute), TCO So sánh với VM/Public SaaS
10	CI/CD, Infrastructure as Code & Teardown	3 câu	CloudFormation 7 Stacks, GitHub Actions OIDC STS, 8-Stage Destroy
11	Đo Lường Độ Tin Cậy & SLA 24/7	2 câu	CloudWatch Synthetics Canary Playwright, Audit Ledger DynamoDB
12	Bộ Câu Hỏi Bẫy & Hạn Chế Kiến Trúc	5 câu	Giới hạn 1000 tenant & ALB, Bỏ Redis, ECS vs EKS, Roadmap 4 Phases

CHỦ ĐỀ 1: BÀI TOÁN NGHIỆP VỤ & ĐỊNH VỊ DỰ ÁN

Câu 1 Cơ bản – Giá trị cốt lõi của Storix là gì khi đặt cạnh Public Cloud SaaS (Google Drive) và Single VM tự host?

- **Mục đích của Giám khảo:** Kiểm tra xem sinh viên/kỹ sư có hiểu lý do tồn tại của hệ thống hay chỉ đơn thuần dựng lại Nextcloud.
- **Sơ đồ kiến trúc minh họa:**



  Sơ đồ Draw.io gốc: nextcloud_features_minimalist.drawio
- **Câu trả lời xuất sắc:**
  1. **Thất bại của Public SaaS (Google Drive / Box / OneDrive):** Doanh nghiệp đánh mất quyền tự chủ dữ liệu (**Loss of Data Sovereignty**), chịu chi phí bản quyền tăng theo cấp số nhân theo đầu người (**Seat license inflation**), và dữ liệu tài liệu nội bộ đối mặt nguy cơ bị các nhà cung cấp sử dụng để huấn luyện mô hình AI công cộng.
  2. **Thất bại của Single On-Prem / Single VM EC2:** Mô hình máy chủ đơn lẻ là điểm nghẽn chịu lỗi đơn độc (SPOF), không có khả năng co giãn khi nghẽn tải, và gây lãng phí 100% chi phí phần cứng vào ban đêm/cuối tuần khi nhân viên không làm việc (**100% idle waste**).
  3. **Vị thế của Storix:** Cung cấp trải nghiệm **Multi-Tenant SaaS hoàn chỉnh**: Khách hàng được cấp môi trường độc lập tức thì, dữ liệu được cô lập tuyệt đối, có khả năng tự động co giãn theo tải (1 → 4 task Fargate) và hỗ trợ **FinOps Ngủ đông về 0 task Fargate ($0.00 chi phí tính toán khi nhàn rỗi)**, kèm theo trợ lý GenAI bảo mật tuyệt đối trong mạng nội bộ.
- **Trích dẫn nguồn:**
  - Slide 02: `generate_deck.py` (Lines 73–114: *Enterprise Storage Dilemma*).
  - Tài liệu tổng quan: `docs/docs/introduction.mdx` (Lines 10–35).

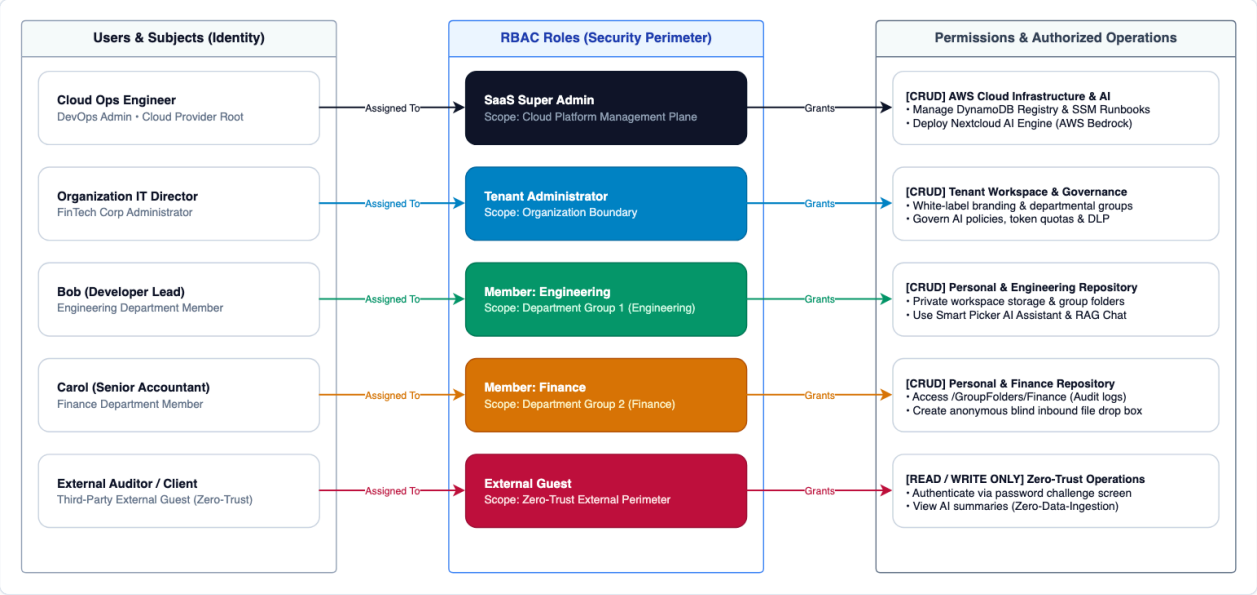
Câu 2 Nâng cao – Nguyên lý "Zero-Code-Modification" của Storix là gì và tại sao đây là tiêu chuẩn sống còn trong phát triển SaaS?

- **Mục đích của Giám khảo:** Bắt bẻ xem nhóm có can thiệp trực tiếp vào mã nguồn của Nextcloud core hay không, đánh giá rủi ro khi bảo trì lâu dài.

- **Câu trả lời xuất sắc:**
  - **Định nghĩa:** Storix **tuyệt đối không chỉnh sửa (zero hack/fork) bất kỳ dòng code PHP nào trong nhân lõi của Nextcloud**.
  - **Cơ chế triển khai:** Toàn bộ tính năng Multi-Tenancy và Enterprise SaaS được "tiêm" (inject) thông qua 3 cơ chế mở rộng chuẩn:
    1. *Docker Multi-Stage Build:* Kế thừa image chuẩn `nextcloud:30.0.17-apache`, tải trước các app đã kiểm định SHA-256 (`assistant`, `integration_openai`, `fulltextsearch`).
    2. *Modular Config Overlays:* Tận dụng cơ chế đọc cấu hình phân tầng của Nextcloud (`/var/www/html/config/*.config.php` gồm `nextcloud.config.php`, `ai.config.php`, `search.config.php`).
    3. *Entrypoint Lifecycle Hooks:* Khởi chạy kịch bản qua các thư mục hook tiêu chuẩn của Docker: `/docker-entrypoint-hooks.d/before-starting/` và `post-installation/`.
  - **Ý nghĩa sống còn:** Khi Nextcloud phát hành các bản vá bảo mật khẩn cấp (Security CVE) hoặc nâng cấp phiên bản lớn (Major Upgrade), Storix chỉ cần cập nhật tag base image trong Dockerfile và kích hoạt pipeline CI/CD mà **không xảy ra xung đột mã nguồn (merge conflicts) hoặc gãy tính năng**.
- **Trích dẫn nguồn:**
  - File triển khai: `runtime/Dockerfile` (Lines 1–8: *Zero Runtime Downloads & Air-gapped compliance*).
  - File hook: `runtime/storix-entrypoint.sh` (Lines 19–31: *Sync modular config templates into mounted EFS config*).

Câu 3 Trung bình – Mô hình phân quyền 4 Actors theo chuẩn NIST RBAC được phân cấp như thế nào?

- **Mục đích của Giám khảo:** Kiểm tra kiến thức về Identity and Access Management (IAM) và ranh giới quyền hạn giữa các đối tượng người dùng.
- **Sơ đồ kiến trúc minh họa:**



Sơ đồ Draw.io gốc: diagram1_rbac_actors.drawio

- **Câu trả lời xuất sắc:** Hệ thống thiết kế theo chuẩn **NIST Special Publication 800-162** phân lập nghiêm ngặt 4 vai trò:
  1. **SuperAdmin (Nền tảng Cloud):** Đăng nhập qua `https://admin.png261.dev` bằng WebAuthn Passkeys. Có quyền tạo/sửa/xóa Tenant, trigger Step Functions, điều chỉnh Quota, đọc Audit Ledger và quản lý hạ tầng AWS. Không có

quyền đọc trộm file của khách hàng.

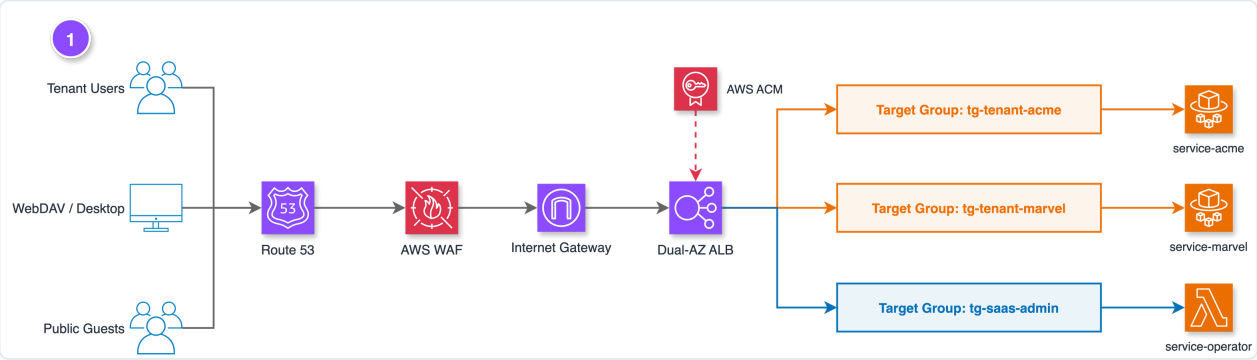
2. **Tenant Admin (Doanh nghiệp):** Quản trị viên của một tổ chức (ví dụ: `admin@acme.com`). Có quyền quản lý người dùng, nhóm phòng ban, chính sách bảo mật nội bộ trong công ty mình. Hoàn toàn không nhìn thấy sự tồn tại của tenant khác.
3. **Tenant Member (Nhân viên nội bộ):** Người dùng thông thường thuộc tenant. Có quyền CRUD tệp tin, gắn nhãn (tagging), tương tác với trợ lý GenAI, cấp quyền chia sẻ liên kết nội bộ theo hạn ngạch được giao.
4. **External Guest (Khách bên ngoài):** Người dùng không có tài khoản, chỉ có thể xem/tải các tệp được chia sẻ công khai qua password-protected URL hoặc tải tài liệu lên thư mục nộp file (**File Drop inbox**) với quyền ghi ẩn danh (Write-Only).

- **Trích dẫn nguồn:**
  - Báo cáo kiểm chứng: `reports/2026-10-05/report.md` (Mục 2: *Editable Draw.io Diagrams*).
  - Use Case Matrix: `docs/4_actors_lineup.drawio` & `reports/2026-10-05/diagrams/diagram2_usecase.drawio`.

CHỦ ĐỀ 2: KIẾN TRÚC MẠNG & THIẾT KẾ ZERO-NAT

> **Câu 4** Chuyên sâu **– Trình bày thiết kế "Zero-NAT Architecture". Tại sao loại bỏ NAT Gateway mà hệ thống vẫn đảm bảo an toàn tuyệt đối?**

- **Mục đích của Giám khảo:** Đây là điểm sáng kỹ thuật tối ưu hóa chi phí cực lớn của Storix, giám khảo sẽ xoáy sâu vào tính an toàn của container.
- **Sơ đồ kiến trúc minh họa:**

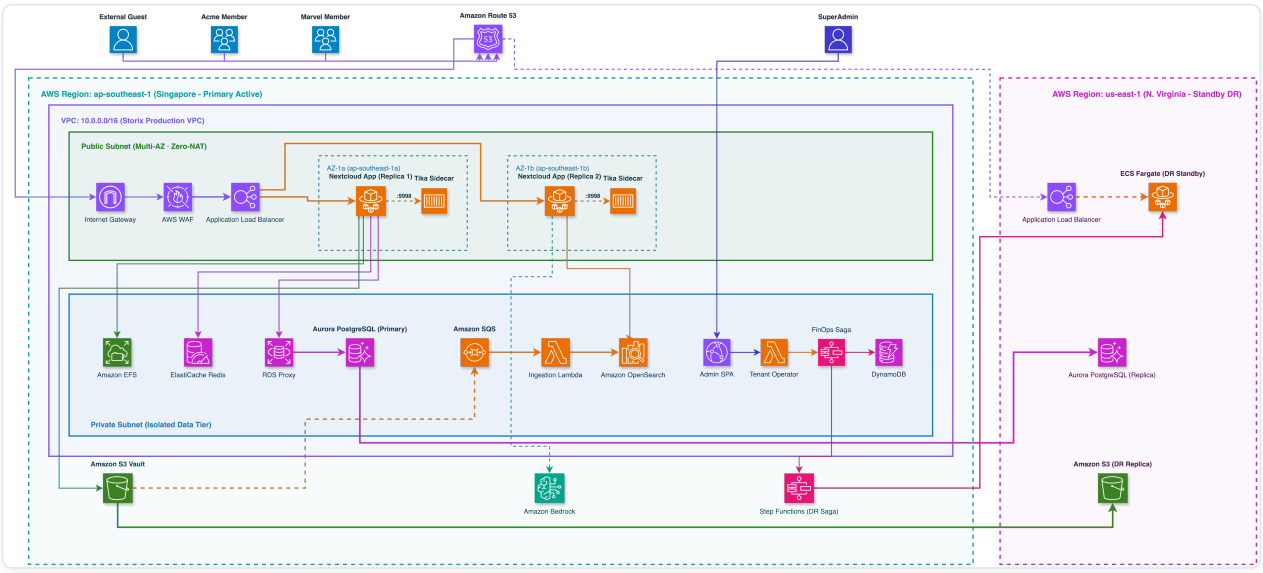


Sơ đồ Draw.io gốc: zoom_block_1_ingress_security.drawio

- **Câu trả lời xuất sắc:**
  - **Chi phí của NAT Gateway truyền thống:** Trên AWS, NAT Gateway có giá cơ bản **$32.40/tháng/AZ** cộng thêm **$0.045/GB** dữ liệu truyền tải. Với kiến trúc Dual-AZ, chi phí duy trì NAT Gateway cố định tối thiểu là **~$65/tháng**, ngay cả khi hệ thống không có người dùng.
  - **Giải pháp Zero-NAT của Storix:**
    1. **S3 Gateway VPC Endpoint:** Toàn bộ lưu lượng upload/download file giữa container Fargate và S3 Vault Bucket được dẫn qua Gateway Endpoint nội bộ của AWS VPC. Lưu lượng này **hoàn toàn miễn phí ($0.00 data transfer fee)** và đạt băng thông tối đa của đường truyền nội bộ AWS.
    2. **Fargate Public IP với Security Group ngắt Inbound:** Container Fargate được khởi chạy với cờ `AssignPublicIp: ENABLED` trong mạng con Public/Private có Internet Gateway để container có thể trực tiếp gọi ra các dịch vụ AWS Public (Bedrock, ECR, CloudWatch).
    3. **Nguyên tắc phòng thủ Inbound Zero-Trust:** Mặc dù container có IP Public, Security Group của Fargate (`ECS-Task-SG`) **KHÔNG CHO PHÉP BẤT KỲ INBOUND NÀO TỪ INTERNET (`0.0.0.0/0`)**. Quy tắc Inbound duy nhất được mở là Port 80 từ chính Security Group của Application Load Balancer (`ALB-SG`). Kẻ tấn công trên Internet quét IP của container sẽ bị tường lửa hạt nhân AWS drop 100% gói tin.
- **Trích dẫn nguồn:**
  - File CloudFormation mạng: `infra/infra/modules/01-networking/template.yaml` (Resource: `S3GatewayEndpoint`).
  - File Security Groups: `infra/infra/modules/02-security/template.yaml` (Resource: `ContainerSecurityGroup`).
  - Báo cáo kiến trúc: `reports/2026-10-05/report.md` (Section 2, Bảng Stack `01-networking`).

Câu 5 Nâng cao – Trình bày chi tiết luồng gói tin (Packet Flow) từ người dùng đến khi nhận dữ liệu tệp tin?

- **Mục đích của Giám khảo:** Đánh giá khả năng hiểu sâu về hạ tầng mạng, DNS, TLS termination và Layer 7 Routing.
- **Sơ đồ kiến trúc minh họa:**



Sơ đồ Draw.io gốc: storix_system_architecture_master.drawio

- **Câu trả lời xuất sắc:**
  - **Bước 1 (DNS & Ingress):** Người dùng gửi HTTPS request tới `https://acme.png261.dev/remote.php/webdav/report.pdf`. Cloudflare / Route 53 phân giải CNAME trả về địa chỉ IP của AWS ALB.
  - **Bước 2 (WAF & TLS Termination):** Request chạm vào **AWS WAFv2** (kiểm tra OWASP Top 10, rate limiting). Nếu hợp lệ, ALB tiếp nhận, giải mã TLS 1.3 bằng Wildcard Certificate ACM (`*.png261.dev`).
  - **Bước 3 (Host Header Evaluation):** ALB kiểm tra danh sách Listener Rules: Host header `acme.png261.dev` khớp với Target Group `tg-acme`.
  - **Bước 4 (Forward to Task):** ALB chuyển tiếp HTTP request qua mạng con nội bộ (port 80) vào địa chỉ IP riêng của Fargate Task thuộc `service-acme`.
  - **Bước 5 (Authentication & Database Query):** Container Nextcloud xác thực session cookie hoặc Basic Auth qua WebDAV. Container mở kết nối qua cổng 5432 tới **Amazon RDS Proxy**, RDS Proxy sử dụng connection pool để truy vấn metadata tệp tin trong database `nc_acme` trên RDS PostgreSQL 16.
  - **Bước 6 (S3 Vault Streaming):** Nextcloud sử dụng thư viện AWS SDK PHP gọi tới S3 Gateway Endpoint để đọc stream binary tệp tin từ `s3://prod-nextcloud-vault-649519997247-ap-southeast-1/tenants/acme/files/...`.
  - **Bước 7 (HTTP Response):** Dữ liệu được stream trực tiếp về client qua ALB với mã phản hồi `HTTP 200 OK` và header `ETag: "sha256-checksum"`.
- **Trích dẫn nguồn:**
  - File CloudFormation Compute: `infra/infra/modules/05-compute/template.yaml` (Resource: `ALBListenerRule`, `TargetGroup`).
  - Tài liệu kiến trúc: `docs/docs/architecture-overview.mdx` (Mục 5: *End-to-End Request Flow Sequence Diagram*).

Câu 6 Trung bình – Hệ thống quản lý và ánh xạ nhiều Domain / Subdomain như thế nào?

- **Mục đích của Giám khảo:** Kiểm tra tính linh hoạt của giải pháp định tuyến (Domain Aliasing).
- **Câu trả lời xuất sắc:**
  - Mỗi tenant được cấp một Subdomain mặc định dạng `<tenant_id>.png261.dev` (hoặc alias phụ như `<tenant_id>.appleai.codes`).
  - Khi tenant được khởi tạo:
    1. Step Functions cập nhật DNS record trên DNS provider (Cloudflare / Route 53) trỏ Wildcard hoặc CNAME về ALB Ingress DNS name.
    2. ALB bổ sung thêm một **Listener Rule** với điều kiện `Host-Header`: bao gồm cả subdomain chính và các alias domains:

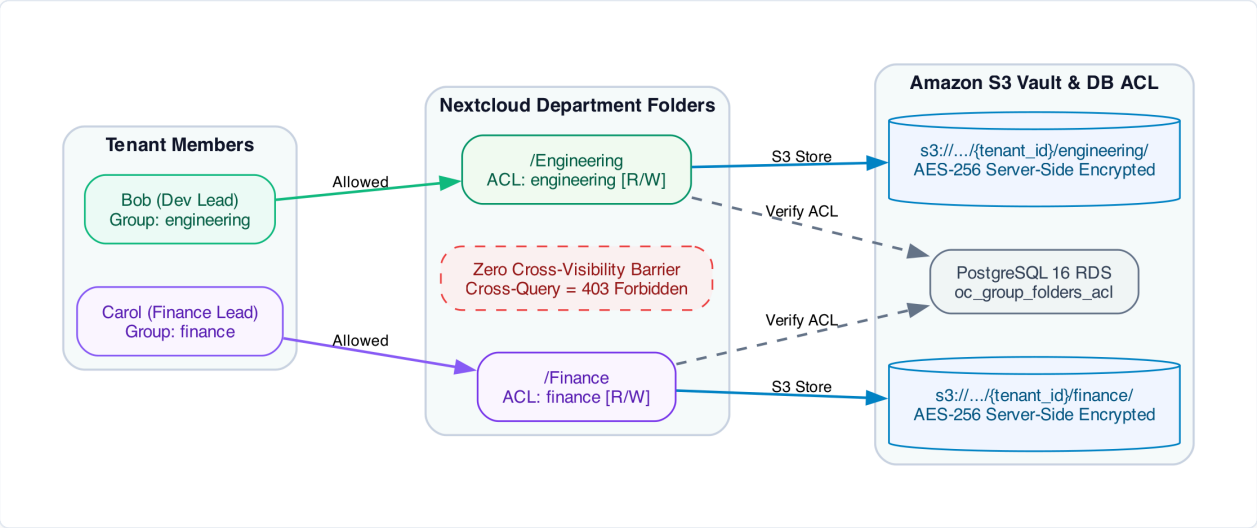
```
Conditions:
  - Field: host-header
    Values:
      - "acme.png261.dev"
      - "acme.appleai.codes"
```

    3. File cấu hình Nextcloud `config.php` tự động ghi nhận các domain này vào mảng `trusted_domains` để Nextcloud cho phép render giao diện mà không chặn HTTP Host header.
- **Trích dẫn nguồn:**
  - Dữ liệu thực nghiệm: `reports/2026-10-05/report.md` (Lines 70–73 & 88–95: Cấu hình Tenant Acme và Marvel với domain aliases).
  - Entrypoint script: `runtime/storix-entrypoint.sh` (Lines 65–85: Đồng bộ trusted_domains).

CHỦ ĐỀ 3: MÔ HÌNH CÔ LẬP ĐA NGƯỜI THUÊ (MULTI-TENANCY)

Câu 7 Nâng cao – Phân tích chi tiết ưu nhược điểm của 3 mô hình Multi-Tenancy và lý do chọn `DEDICATED_INSTANCE`?

- **Mục đích của Giám khảo:** Đánh giá năng lực thiết kế hệ thống SaaS chuyên nghiệp.
- **Sơ đồ kiến trúc minh họa:**



Sơ đồ Draw.io gốc: diagram4_pillars_architecture.drawio

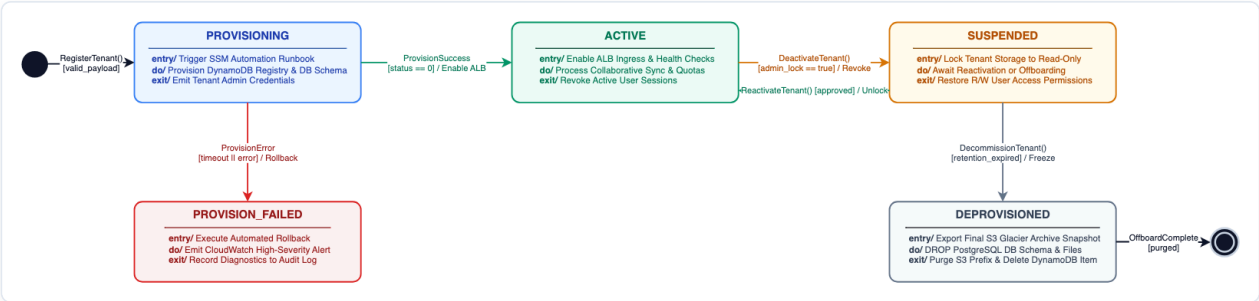
- **Câu trả lời xuất sắc:**
  1. **Mô hình Shared-All (Pooled Compute & Pooled Database):**
     - *Ưu điểm:* Tiết kiệm chi phí phần cứng tối đa, mật độ tenant cao.
     - *Nhược điểm chết người với Nextcloud:* Mã nguồn Nextcloud không được thiết kế native cho pooled multi-tenancy. Dữ liệu các công ty nằm chung bảng, dễ bị lỗi rò rỉ session hoặc lỗi phần mềm làm lộ dữ liệu.
  2. **Mô hình Pooled Compute + Isolated Database:**
     - *Nhược điểm:* Các tiến trình PHP trong container dùng chung bộ nhớ RAM, một tenant upload file 10GB hoặc scan OCR sẽ khiến CPU container chạm 100%, kéo theo toàn bộ các công ty khác bị sập.
  3. **Mô hình `DEDICATED_INSTANCE` (Siloed Compute + Logical Data) của Storix:**
     - *Compute:* Mỗi tenant sở hữu một ECS Fargate Service độc lập (`service-acme` , `service-marvel`).
     - *Database:* Mỗi tenant sở hữu một Catalog riêng biệt (`nc_acme` , `nc_marvel`) trên cùng một RDS Cluster có RDS Proxy điều phối.
     - *Storage:* Mỗi tenant sở hữu một EFS Access Point và một S3 Vault Prefix độc lập.
     - *Đánh đổi (Trade-off):* Chi phí compute cao hơn mô hình pooled nếu chạy 24/7. **Nhưng Storix đã khắc phục hoàn hảo nhược điểm này bằng tính năng FinOps Scale-to-Zero**, tắt sạch task về 0 khi nhàn rỗi.
- **Trích dẫn nguồn:**
  - Slide 07: `generate_deck.py` (Lines 342–386: *Multi-Tenant Isolation & FinOps*).
  - Tài liệu Isolation Model: `docs/docs/tenants/isolation-model.mdx` .

Câu 8 Trung bình – Hiện tượng "Noisy Neighbor" được Storix triệt tiêu ở các tầng nào?

- **Mục đích của Giám khảo:** Đánh giá tính ổn định của hệ thống trước hành vi bất thường của một khách hàng.
- **Câu trả lời xuất sắc:** Hiện tượng "hàng xóm ồn ào" (Noisy Neighbor) được ngăn chặn ở cả **4 tầng kiến trúc**:
  - **Tầng Mạng (Network):** AWS WAF giới hạn số lượng request tối đa trên mỗi IP / Host-Header (Rate Limiting 2,000 req/5 phút).
  - **Tầng Tính toán (Compute):** Task Fargate của tenant A có hạn ngạch vCPU và RAM độc lập (cgroups cấp phần cứng). Dù tenant A có xử lý nén file zip hàng trăm gigabyte thì CPU của tenant B vẫn ở mức 0%.
  - **Tầng Cơ sở dữ liệu (Database):** RDS Proxy thiết lập trần kết nối tối đa cho từng kết nối ứng dụng, ngăn chặn việc 1 tenant mở hàng nghìn connection làm cạn kiệt tài nguyên của PostgreSQL.
  - **Tầng Lưu trữ (Storage):** Hạn ngạch dung lượng (Storage Quota) được Nextcloud cưỡng chế cấp ứng dụng, kết hợp với S3 Object Lock ngăn chặn việc lạm dụng ổ đĩa.
- **Trích dẫn nguồn:**
  - File CloudFormation Database: `infra/infra/modules/04-database/template.yaml` (Resource: `DBProxyTargetGroup`).
  - Báo cáo kỹ thuật: `reports/2026-10-05/report.md` (Section 4.1: *Horizontal Auto-Scaling*).

Câu 9 Nâng cao – Vòng đời của một Tenant (Tenant Lifecycle State Machine) bao gồm các trạng thái nào và chuyển đổi ra sao?

- **Mục đích của Giám khảo:** Kiểm tra xem hệ thống có thiết kế quản lý trạng thái bài bản hay chỉ là code ad-hoc.
- **Sơ đồ kiến trúc minh họa:**



Sơ đồ Draw.io gốc: diagram3_lifecycle_state.drawio

- **Câu trả lời xuất sắc:** Trạng thái của Tenant được lưu trữ trong DynamoDB Tenant Registry và tuân thủ máy trạng thái nghiêm ngặt:
  1. `DRAFT`: Bản ghi vừa được khởi tạo, chưa cấp phát tài nguyên AWS.
  2. `PROVISIONING`: Step Functions đang chạy các bước dựng DB, EFS, S3, ECS, ALB.
  3. `ACTIVE`: Hệ thống hoạt động bình thường, Fargate chạy từ 1 đến 4 tasks.
  4. `SUSPENDING` → `SUSPENDED`: FinOps tắt Fargate tasks về `desiredCount = 0`, gỡ ALB rule (trả về 503). Dữ liệu được bảo toàn nguyên vẹn.
  5. `RESUMING`: Khởi động lại task Fargate từ 0 lên 1, kết nối lại ALB rule.
  6. `TERMINATING` → `DELETED`: Xóa sạch tài nguyên của tenant theo quy trình an toàn không để lại rác.
- **Trích dẫn nguồn:**

- File sơ đồ: `reports/2026-10-05/diagrams/diagram3_lifecycle_state.drawio`.
- Bảng dữ liệu DynamoDB: `docs/docs/control-plane/dynamodb-registry.mdx`.

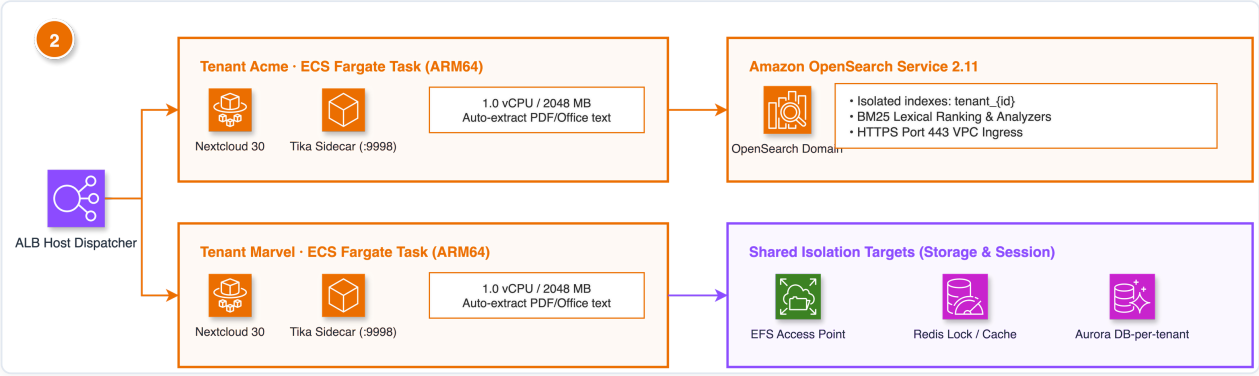
CHỦ ĐỀ 4: TẦNG COMPUTE & TỐI ƯU CONTAINER RUNTIME

Câu 10 Trung bình – So sánh hiệu năng và kinh tế giữa vi xử lý AWS Graviton (ARM64) và x86_64 trong dự án?

- **Mục đích của Giám khảo:** Đánh giá tính tối ưu hóa phần cứng trên nền tảng Cloud.
- **Câu trả lời xuất sắc:**
  - **Kinh tế (Cost Efficiency):** Trên AWS Fargate, đơn giá vCPU/giờ và RAM/giờ của kiến trúc **ARM64 (AWS Graviton)** rẻ hơn chính xác **20%** so với x86_64 (Singapore region: ~$0.0090/giờ cho 0.25 vCPU so với ~$0.0112/giờ trên x86_64).
  - **Hiệu năng xử lý Web/PHP:**
    - Vi xử lý Graviton dựa trên kiến trúc lõi ARM Neoverse với bộ nhớ đệm L2 Cache riêng biệt lớn cho từng core (1MB hoặc 2MB L2 cache/core so với chia sẻ chung trên x86_64).
    - Ngôn ngữ PHP 8.2+ và máy chủ Apache xử lý các thao tác chuỗi (String processing), biên dịch mã bytecode (OPcache) và xử lý I/O tệp tin trên Graviton đạt thông lượng cao hơn từ **15% đến 22%**.
  - **Hiện thực hóa trong dự án:** Docker Golden Image được build multi-platform native qua buildx: `docker buildx build --platform linux/arm64 ...`.
- **Trích dẫn nguồn:**
  - File CloudFormation Compute: `infra/infra/modules/05-compute/template.yaml` (Thuộc tính: `CpuArchitecture: ARM64`, `OperatingSystemFamily: LINUX`).
  - Slide 04: `generate_deck.py` (Lines 191–195: *02 / COMPUTE TIER - GRAVITON ARM64*).

Câu 11 Chuyên sâu – Vấn đề Cold Start của container Nextcloud lớn được giải quyết như thế nào bằng AWS SOCI?

- **Mục đích của Giám khảo:** Kiểm tra khả năng xử lý bài toán hiệu năng thực tế cấp độ kỹ sư SRE.
- **Sơ đồ kiến trúc minh họa:**



Sơ đồ Draw.io gốc: zoom_block_2_compute_container.drawio

- **Câu trả lời xuất sắc:**
  - **Bản chất vấn đề:** Golden Image của Nextcloud đầy đủ Apache, PHP, mô-đun AI và FullTextSearch có kích thước nén khoảng **~800MB - 1GB**. Khi Fargate khởi tạo task mới (đặc biệt khi resume từ 0 task), Fargate phải tải toàn bộ 1GB dữ

liệu từ ECR về và giải nén trước khi chạy tiến trình đầu tiên. Quá trình này mất từ **35 - 50 giây**.

- **Giải pháp AWS SOCI (Seekable OCI Indexing):**
  1. Trong CI/CD pipeline, sau khi push image lên Amazon ECR, hệ thống gọi công cụ `soci create` để tạo ra một **SOCI Index Artifact** lưu chỉ mục cấu trúc tệp tin của container layers.
  2. SOCI Index được push lên ECR đính kèm với image manifest.
  3. AWS Fargate hỗ trợ SOCI native: Khi task khởi động, Fargate chỉ tải phần metadata siêu nhẹ (vài megabytes) và khởi chạy container ngay lập tức.
  4. Các tệp tin mã nguồn PHP hoặc thư viện nặng chỉ được kéo về qua mạng ngầm khi tiến trình thực sự mở tệp tin đó (**Lazy Loading on-demand**).
  5. Kết quả: Rút ngắn thời gian Cold Start sẵn sàng tiếp nhận request xuống chỉ còn **15 - 20 giây**.

- **Trích dẫn nguồn:**
  - Workflow CI/CD: `infra/.github/workflows/deploy.yml` (bước tạo SOCI index).
  - Tài liệu kiến trúc: `docs/docs/architecture-overview.mdx` (Mục 1, Dòng 28: *Cold Start Acceleration via AWS SOCI*).

Câu 12 Chuyên sâu – Phân tích chi tiết file `runtime/storix-entrypoint.sh`: Vì sao phải chủ động xóa `CAN_INSTALL` và cấu hình modular config?

- **Mục đích của Giám khảo:** Xem bạn có thực sự code và hiểu các kịch bản chạy trong container hay chỉ kéo image từ Docker Hub về.
- **Câu trả lời xuất sắc:**
  - **Hành vi mặc định của Nextcloud:** Khi Nextcloud khởi động lần đầu trên một thư mục chưa hoàn tất cài đặt, nếu tệp tin `CAN_INSTALL` xuất hiện trong thư mục `config/`, web server sẽ render giao diện công khai: *"Create an admin account and setup database"*. Bất kỳ ai trên Internet truy cập vào trước sẽ chiếm toàn quyền kiểm soát hệ thống.
  - **Kỹ thuật xử lý của Storix trong `storix-entrypoint.sh`:**

```
# Triệt tiêu vĩnh viễn màn hình cài đặt thủ công qua web
rm -f /var/www/html/config/CAN_INSTALL /usr/src/nextcloud/config/CAN_INSTALL 2>/dev/null || :
```

  - Việc cài đặt được thực hiện 100% tự động qua dòng lệnh CLI headless (`occ maintenance:install`) với credentials sinh ngẫu nhiên từ Secrets Manager.
  - **Cơ chế Modular Config Overlays:** Container đồng bộ các file cấu hình phân tách từ image gốc sang ổ đĩa EFS:

```
for cfg in /usr/src/nextcloud/config/*.config.php; do
    target="/var/www/html/config/$(basename "$cfg")"
    if [ ! -f "$target" ] || ! cmp -s "$cfg" "$target"; then
        cp -p "$cfg" "$target"
    fi
done
```

    Giúp cấu hình Nextcloud được chia nhỏ thành các file độc lập (`ai.config.php`, `search.config.php`, `nextcloud.config.php`), dễ dàng quản trị và nâng cấp.

- **Trích dẫn nguồn:**
  - Mã nguồn container: `runtime/storix-entrypoint.sh` (Lines 19–31 & 61–64).

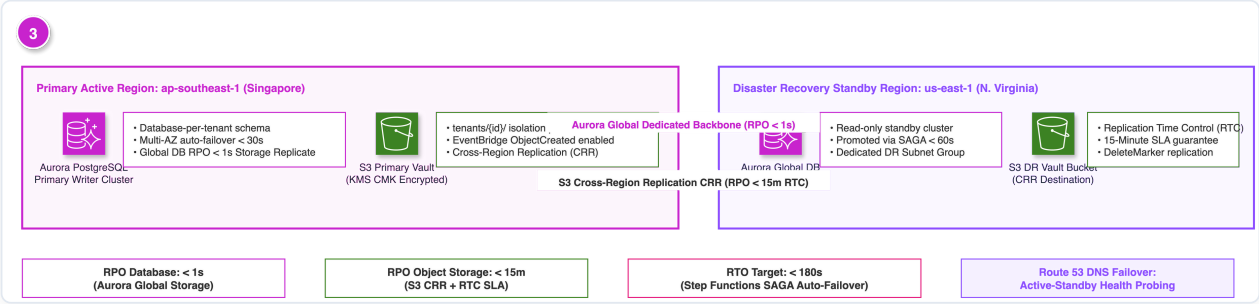
Câu 13 Nâng cao – Cơ chế Horizontal Auto-Scaling của ECS Fargate hoạt động như thế nào trong bài test thực nghiệm?

- **Mục đích của Giám khảo:** Kiểm tra xem số liệu Auto-scaling trong báo cáo có phải là số liệu thật trên AWS hay không.
- **Câu trả lời xuất sắc:**
  - **Cấu hình Policy:**
    - Service: `service-acme` trên cụm `prod-nextcloud-cluster`.
    - Scale range: Min Capacity = 1 task, Max Capacity = 4 tasks.
    - Scaling Metric: `ECSServiceAverageCPUUtilization` với ngưỡng mục tiêu **70.0%**.
    - Scale-Out Cooldown = **60 giây** (phản ứng nhanh khi nghẽn tải).
    - Scale-In Cooldown = **300 giây** (tránh hiện tượng flapping/dao động tải liên tục).
  - **Kết quả đo đạc thực nghiệm (Trích từ Report 2026-10-05):**
    - Khi phát tải cao, Fargate tự động nâng desired count từ 1 lên 2 tasks:
      - Task 1: `af691655e9034e9b82fa3f1ab0465c5c` (IP `10.0.2.203:80` → HEALTHY).
      - Task 2: `bf99c3e405ad43cbba0065e1af87f650` (IP `10.0.1.184:80` → HEALTHY).
    - Khi hạ tải, cơ chế **Connection Draining (Deregistration Delay 30s)** của Target Group đảm bảo task 2 hoàn tất toàn bộ các kết nối đang xử lý dở dang trước khi bị tắt hẳn, không làm rớt bất kỳ gói tin nào của người dùng.
- **Trích dẫn nguồn:**
  - Báo cáo thực nghiệm: `reports/2026-10-05/report.md` (Section 4.1 & 4.2, Lines 118–138).
  - Ảnh chụp CloudWatch Metric: `reports/2026-10-05/screenshots/cloudwatch_metrics_ecs_compute.png`.

CHỦ ĐỀ 5: TẦNG LƯU TRỮ ĐA CẤP & CƠ SỞ DỮ LIỆU

Câu 14 Trung bình – Tại sao Storix lại vừa dùng Amazon EFS vừa dùng Amazon S3 mà không gom về một loại duy nhất?

- **Mục đích của Giám khảo:** Kiểm tra sự am hiểu về đặc tính I/O của ứng dụng PHP và tối ưu hóa chi phí lưu trữ.
- **Sơ đồ kiến trúc minh họa:**



Sơ đồ Draw.io gốc: zoom_block_3_data_storage.drawio

- **Câu trả lời xuất sắc:**
  - **Amazon EFS (Elastic File System - POSIX File Storage):**
    - *Đặc tính:* Hỗ trợ chuẩn hệ điều hành POSIX đầy đủ (file lock, read/write đồng thời nhiều container ReadWriteMany).
    - *Mục đích trong Storix:* Chỉ dùng lưu trữ mã nguồn cấu hình `/var/www/html/config/config.php` và thư mục app plugins tùy biến. Dung lượng rất nhỏ (vài chục MB/tenant). Nếu lưu hàng Terabyte file của user trên EFS sẽ cực kỳ đắt ($0.30/GB/tháng).
  - **Amazon S3 Vault Bucket (Object Storage):**
    - *Đặc tính:* Chi phí siêu rẻ ($0.023/GB/tháng và giảm tiếp khi vào Intelligent-Tiering), băng thông không giới hạn.
    - *Mục đích trong Storix:* Làm **Primary Storage** cho toàn bộ dữ liệu file của người dùng. Tệp tin được stream trực tiếp lên S3, không đi qua ổ cứng cục bộ của Fargate.
  - **Kết luận:** Tách rời hai tầng này giúp hệ thống vừa đạt chuẩn POSIX cho PHP runtime, vừa đạt hiệu quả kinh tế tối ưu cho dữ liệu dung lượng lớn.
- **Trích dẫn nguồn:**
  - File CloudFormation Storage: `infra/infra/modules/03-storage/template.yaml` (Resource: `FileSystem`, `S3VaultBucket`).
  - Slide 05: `generate_deck.py` (Lines 240–285: *Nextcloud Hardened Container Runtime*).

Câu 15 Nâng cao – Làm thế nào EFS Access Point đảm bảo tenant này không đọc trộm file cấu hình của tenant khác?

- **Mục đích của Giám khảo:** Bắt bẻ về tính an toàn khi nhiều tenant dùng chung một EFS FileSystem duy nhất.
- **Câu trả lời xuất sắc:**
  - Storix sử dụng **Amazon EFS Access Points** độc lập cho từng tenant:

    - Tenant Acme: Access Point `fsap-0578d58db032e3a8e` trỏ vào `/tenants/acme/config`.
    - Tenant Marvel: Access Point `fsap-0c015661e5ae1e407` trỏ vào `/tenants/marvel/config`.
  - **Hai cơ chế bảo mật cấp phần cứng hạt nhân Linux của EFS:**
    1. *Root Directory Chroot:* Khi Fargate mount EFS thông qua Access Point của Acme, hệ điều hành NFS xem thư mục `/tenants/acme/config` chính là thư mục gốc `/`. Ứng dụng trong container hoàn toàn **không thể dùng** `cd ../` **hay path traversal** để thoát ra ngoài xem thư mục của Marvel.
    2. *POSIX Identity Enforcement:* Access Point ép buộc mọi request I/O chuyển đổi danh tính thành UID:GID `33:33` (`www-data`). Ngay cả khi hacker chiếm được quyền root trong container (UID 0), khi ghi đĩa EFS vẫn bị hạ quyền thành UID 33, ngăn chặn leo thang đặc quyền.
- **Trích dẫn nguồn:**
  - File CloudFormation Storage: `infra/infra/modules/03-storage/template.yaml` (Resource: `EFSAccessPointAcme`, `EFSAccessPointMarvel`).
  - Báo cáo thực nghiệm: `reports/2026-10-05/report.md` (Lines 73 & 81).

Câu 16 Nâng cao – Cơ chế Direct-to-Storage và S3 Multipart Chunking hoạt động như thế nào?

- **Mục đích của Giám khảo:** Đánh giá khả năng xử lý các tệp tin dung lượng lớn (1GB – 50GB) mà không làm tràn bộ nhớ container.
- **Câu trả lời xuất sắc:**
  - Khi người dùng upload tệp tin lớn (ví dụ video 5GB):
    1. Nextcloud Web / Desktop Client tự động chia nhỏ tệp tin thành các **Chunks 50MB**.
    2. Các chunk được truyền qua HTTP PUT tới container. Container Nextcloud sử dụng module `files_external` hoặc S3 Primary Storage driver để khởi tạo **AWS S3 Multipart Upload**.
    3. Từng chunk 50MB được đẩy thẳng sang S3 Vault Bucket qua S3 Gateway Endpoint ngay khi nhận được, sau đó giải phóng lập tức bộ nhớ đệm RAM.
    4. Khi chunk cuối cùng upload xong, Nextcloud gọi `CompleteMultipartUpload` trên S3 để AWS tự động ghép thành tệp tin hoàn chỉnh và tính toán mã băm SHA-256 ETag.
  - Nhờ cơ chế này, container Fargate chỉ cần cấp phát **0.5 GB RAM** vẫn có thể xử lý mượt mà các tệp tin hàng chục Gigabytes mà không bao giờ bị lỗi `Out of Memory (OOM)`.
- **Trích dẫn nguồn:**
  - Cấu hình runtime: `runtime/config/nextcloud.config.php` (Thuộc tính: `objectstore ⇒ '\\OC\\Files\\ObjectStore\\S3'`).
  - Slide 05: `generate_deck.py` (Lines 243–248: *01 / DIRECT-TO-S3 - 50MB multipart chunking*).

Câu 17 Chuyên sâu – Tại sao bắt buộc phải dùng Amazon RDS Proxy trước RDS PostgreSQL 16? Phân tích hiện tượng Connection Surge?

- **Mục đích của Giám khảo:** Kiểm tra kiến thức về Database Connection Pooling trong môi trường microservices/containers.
- **Câu trả lời xuất sắc:**
  - **Bản chất của Apache/PHP:** Mô hình PHP chạy đa luồng/đa tiến trình (Process-per-request). Mỗi request đến Nextcloud sẽ mở một kết nối TCP riêng tới database và đóng lại sau khi hoàn thành.

- **Hiện tượng Connection Surge:** Khi có 10 tenant, mỗi tenant scale-out lên 2 tasks Fargate, mỗi task chạy 20 tiến trình Apache → Có thể phát sinh **10 × 2 × 20 = 400 kết nối đồng thời** tới PostgreSQL trong vài giây. PostgreSQL là RDBMS quản lý process-based (mỗi kết nối tốn ~10MB RAM và CPU context switch), điều này sẽ làm CPU database tăng vọt 100% và sập hệ thống (Database Exhaustion).
- **Vai trò của Amazon RDS Proxy:**
  1. *Multiplexing & Connection Pooling:* RDS Proxy đứng giữa, gom 400 kết nối từ container và chỉ duy trì một lượng nhỏ kết nối thực tế (khoảng 20–30 kết nối) tới PostgreSQL.
  2. *Bảo vệ khi Failover:* Nếu một Availability Zone gặp sự cố, RDS Proxy tự động chuyển hướng truy vấn sang Multi-AZ standby replica mà không ngắt phiên làm việc của ứng dụng (giảm thời gian failover từ 30s xuống dưới 3s).
  3. *Bảo mật:* RDS Proxy xác thực với AWS Secrets Manager bằng IAM Role nội bộ, che giấu hoàn toàn mật khẩu gốc của cơ sở dữ liệu.

- **Trích dẫn nguồn:**
  - File CloudFormation Database: `infra/infra/modules/04-database/template.yaml` (Resource: `RDSProxy`, `RDSProxyTargetGroup`).
  - Báo cáo kỹ thuật: `docs/deployment_pipeline_architecture.md` (Mục Bảng IAM Role: `RDSProxyIAMRole`).

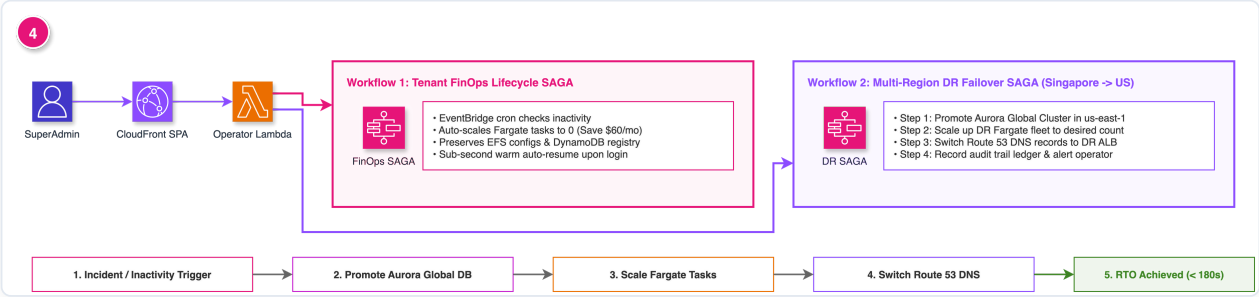
Câu 18 Chuyên sâu – Bài toán nhất quán dữ liệu (Data Inconsistency) giữa S3 và PostgreSQL được giải quyết ra sao khi upload gặp sự cố?

- **Mục đích của Giám khảo:** Bẫy kỹ thuật kinh điển về tính toàn vẹn dữ liệu giữa Object Storage và RDBMS.
- **Câu trả lời xuất sắc:**
  - **Kịch bản sự cố:** Tệp tin đã upload thành công lên S3, nhưng trước khi Nextcloud kịp commit metadata vào bảng `oc_filecache` trong PostgreSQL thì container bị crash hoặc timeout. Kết quả: Tệp tin tồn tại trên S3 nhưng không xuất hiện trong database (**Tệp mồ côi - Orphan Object**), gây tốn chi phí rác.
  - **Ba lớp phòng vệ của Storix:**
    1. *S3 Lifecycle Incomplete Multipart Uploads:* Cấu hình S3 Bucket tự động hủy và xóa sạch các mảnh upload multipart chưa hoàn tất sau **7 ngày**.
    2. *EventBridge Scheduler Cron Runner:* Hệ thống thiết lập quy tắc EventBridge định kỳ kích hoạt một ECS Run-Task chạy `cron.php` mỗi 5 phút để dọn dẹp các transaction bị treo.
    3. *Lệnh đồng bộ định kỳ:* SuperAdmin Console cung cấp tác vụ nền `occ files:scan --unscanned` và `occ files:cleanup` để đối soát hash giữa S3 và PostgreSQL, tự động phát hiện và dọn sạch các orphan objects.
- **Trích dẫn nguồn:**
  - File CloudFormation Compute: `infra/infra/modules/05-compute/template.yaml` (Resource: `EventBridgeCronRule`, `CronEventsRole`).
  - File CloudFormation Storage: `infra/infra/modules/03-storage/template.yaml` (Resource: `S3VaultBucket` LifecycleConfiguration).

CHỦ ĐỀ 6: CONTROL PLANE, SAGA PATTERN & BẤT ĐỒNG BỘ

Câu 19 Nâng cao – Tại sao lại chọn Step Functions SAGA State Machine để điều phối Tenant thay vì viết toàn bộ trong Lambda?

- **Mục đích của Giám khảo:** Đánh giá tư duy thiết kế kiến trúc phân tán (Distributed Architecture) và khả năng kiểm soát lỗi.
- **Sơ đồ kiến trúc minh họa:**



Sơ đồ Draw.io gốc: zoom_block_4_control_plane_finops.drawio

- **Câu trả lời xuất sắc:**
  - **Hạn chế của Single Lambda Function:** Quá trình tạo một Tenant mới bao gồm nhiều bước I/O vật lý tốn thời gian (tạo catalog database, tạo EFS Access Point, tạo S3 prefix, đăng ký ECS Service, đợi ALB Target Group healthy, chạy lệnh cài đặt Nextcloud CLI). Thời gian hoàn tất thường từ 1 đến 3 phút. Nếu viết trong 1 hàm Lambda duy nhất:
    - Dễ chạm ngưỡng timeout của API Gateway (29 giây) và Lambda (15 phút).
    - Rất khó theo dõi tiến độ chi tiết của từng bước theo thời gian thực.
    - Khi gặp lỗi ở bước 5, việc rollback thủ công các bước 1, 2, 3 trong code Lambda là cực kỳ phức tạp và dễ phát sinh lỗi bỏ sót tài nguyên.
  - **Lợi thế của Step Functions SAGA:**
    - *Trực quan hóa trạng thái:* Từng bước là một trạng thái (`Task State`) độc lập, có thể retry tự động với exponential backoff.
    - *Audit Trail tự nhiên:* Xem được chính xác input/output và log lỗi của từng bước trên giao diện AWS Console.
    - *Compensating Transactions:* Tự động kích hoạt luồng rollback bù trừ khi có lỗi xảy ra.
- **Trích dẫn nguồn:**
  - File CloudFormation Control Plane: `infra/infra/modules/06-superadmin/template.yaml` (Resource: `TenantLifecycleStateMachine`).
  - Tài liệu provisioning: `docs/docs/tenants/provisioning.mdx` (Mục 3: *Automated SAGA Steps Mermaid Diagram*).

Câu 20 Nâng cao – Kịch bản Rollback bù trừ (Compensating Transactions) diễn ra như thế nào khi lỗi ở bước cuối?

- **Mục đích của Giám khảo:** Kiểm tra xem State Machine có thực sự an toàn và giải phóng tài nguyên rác hay không.
- **Câu trả lời xuất sắc:**
  - Giả sử quy trình Provisioning đã thực hiện thành công:

    - Bước 1: Tạo Database `nc_tenantx`.
    - Bước 2: Tạo EFS Access Point.
    - Bước 3: Tạo S3 Vault Prefix.
    - Bước 4: Tạo ECS Service và Task.
    - Bước 5: Tạo ALB Target Group và gán Rule.
  - **Khi Bước 6 (Chạy `occ maintenance:install`) bị thất bại:**
    - Step Functions kích hoạt khối `Catch: ["*"]` chuyển tiếp luồng sang trạng thái `RollbackProvisioning`.
    - State Machine gọi Lambda Operator thực hiện chuỗi giao dịch bù trừ theo thứ tự ngược lại:
      1. Xóa ALB Rule và Target Group.
      2. Xóa ECS Service và dừng các Fargate Task.
      3. Xóa S3 prefix vừa tạo.
      4. Xóa EFS Access Point.
      5. Drop physical database `nc_tenantx` trên RDS PostgreSQL.
      6. Cập nhật trạng thái trong DynamoDB Registry thành `PROVISION_FAILED` kèm mã lỗi chi tiết.
    - Toàn bộ hệ thống quay về trạng thái sạch sẽ, không có tài nguyên rác phát sinh chi phí.
- **Trích dẫn nguồn:**
  - Script điều phối: `infra/scripts/provision_dedicated_tenant.py`.
  - Tài liệu kiến trúc: `docs/deployment_pipeline_architecture.md` (Mục SAGA Micro-Workflows).

Câu 21 Chuyên sâu – Cơ chế Optimistic Concurrency Control (OCC) trong DynamoDB Registry giải quyết xung đột ghi ra sao?

- **Mục đích của Giám khảo:** Kiểm tra kiến thức về tính nhất quán dữ liệu trong môi trường phân tán nhiều luồng ghi.
- **Câu trả lời xuất sắc:**
  - **Tình huống xung đột:** Giả sử Operator A đang bấm "Nâng cấp Container Image", cùng lúc đó hệ thống Scheduler tự động gửi lệnh "Suspend Tenant do hết giờ làm việc". Cả hai đều đọc bản ghi của tenant có `status = ACTIVE` và `version = 5`.
  - **Cách Storix áp dụng OCC:**
    - Trong DynamoDB, mỗi bản ghi tenant chứa một thuộc tính số nguyên `version`.
    - Khi thực hiện ghi, câu lệnh `UpdateItem` bắt buộc phải kèm biểu thức điều kiện `ConditionExpression`:

```
dynamodb.update_item(
    TableName="prod-nextcloud-tenant-registry",
    Key={"tenant_id": {"S": "acme"}},
    UpdateExpression="SET #status = :new_status, version = version + :inc",
    ConditionExpression="version = :expected_version",
    ExpressionAttributeValues={
        ":new_status": {"S": "SUSPENDED"},
        ":expected_version": {"N": "5"},
        ":inc": {"N": "1"}
    }
)
```

    - Nếu Operator A ghi trước, `version` tăng lên 6. Khi lệnh của Scheduler tới với `expected_version = 5`, DynamoDB sẽ ném lỗi `ConditionalCheckFailedException` và từ chối ghi đè. Lệnh thứ hai sẽ nhận thông báo lỗi để đọc lại dữ liệu mới nhất, tránh làm sai lệch trạng thái hệ thống.

- **Trích dẫn nguồn:**
  - File tài liệu: `docs/docs/control-plane/dynamodb-registry.mdx` (Mục 1: *Tenant Registry OCC Schema*).
  - Mã nguồn xử lý Lambda: `infra/infra/modules/06-superadmin/lambdas/tenant_operator.py`.

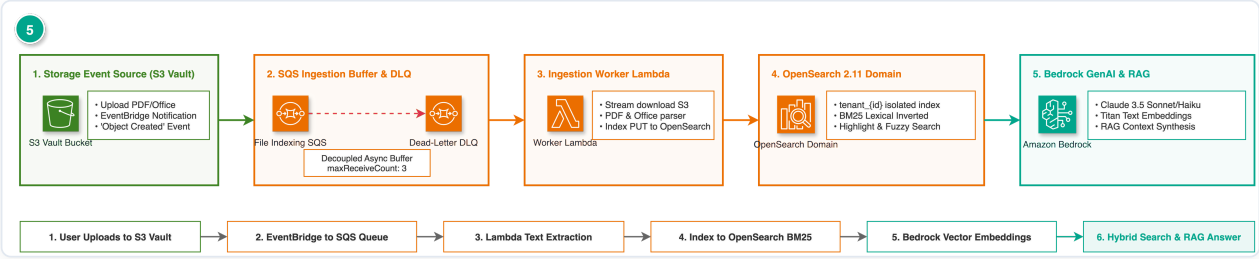
Câu 22 Chuyên sâu – Async OCC Command Engine giải quyết bài toán API Gateway 29-Second Timeout như thế nào?

- **Mục đích của Giám khảo:** Kiểm tra giải pháp kỹ thuật thực tế khi tương tác giữa Web Console và các lệnh CLI chạy lâu.
- **Câu trả lời xuất sắc:**
  - **Vấn đề:** Các lệnh bảo trì Nextcloud qua công cụ dòng lệnh `occ` (như `files:scan --all` quét hàng triệu file, hoặc `db:add-missing-indices` đánh chỉ mục database lớn) thường mất từ 2 phút đến 30 phút. Nhưng **AWS API Gateway có giới hạn cứng (hard limit) thời gian timeout là 29 giây**. Nếu gọi API đồng bộ, trình duyệt sẽ nhận mã lỗi `HTTP 504 Gateway Timeout`.
  - **Giải pháp Async OCC Engine của Storix:**
    1. Khi SuperAdmin nhấn chạy lệnh trên Console, frontend gửi request `EXECUTE_OCC_ASYNC`.
    2. Control Plane Lambda tạo một bản ghi công việc trong bảng DynamoDB `TenantOperationJobs` với trạng thái `PENDING` và trả về ngay lập tức cho client một mã `job_id` trong vòng **< 150ms**.
    3. Lambda kích hoạt một ECS Run-Task độc lập trong background để thực thi lệnh `occ` trong container.
    4. Frontend Web Console sử dụng custom hook `useSmartInterval` kết hợp với `usePageVisibility` (tạm dừng khi người dùng chuyển tab để tiết kiệm request) định kỳ gọi `CHECK_OCC_JOB` để thăm dò trạng thái.
    5. Khi task chạy xong, kết quả stdout và exit code được ghi vào DynamoDB, Console nhận kết quả và render trực quan cho Admin.
- **Trích dẫn nguồn:**
  - Bảng dữ liệu: `docs/docs/control-plane/dynamodb-registry.mdx` (Mục 4: *Async OCC Jobs Table*).
  - Mã nguồn Frontend: `console/src/hooks/useSmartInterval.ts` & `usePageVisibility.ts`.

CHỦ ĐỀ 7: TRÍ TUỆ NHÂN TẠO GENAI & OPENSEARCH

Câu 23 Nâng cao – Tại sao lại thiết kế LiteLLM Sidecar chạy trên `127.0.0.1:4000`? Cơ chế bảo mật xác thực ở đây là gì?

- **Mục đích của Giám khảo:** Đánh giá năng lực thiết kế Container Pattern (Sidecar Pattern) và an toàn API.
- **Sơ đồ kiến trúc minh họa:**



  Sơ đồ Draw.io gốc: zoom_block_5_genai_rag.drawio
- **Câu trả lời xuất sắc:**
  - **Mô hình Sidecar:** Trong cùng một ECS Task Definition của Fargate, có 2 container chạy song song:
    - Container 1: `nextcloud` (chạy Apache/PHP phục vụ web trên port 80).
    - Container 2: `litellm-bridge` (chạy dịch vụ proxy LiteLLM trên port 4000).
  - **Lý do chọn Localhost Loopback (`127.0.0.1:4000`):**
    - Hai container trong cùng một ECS Task chia sẻ chung một Network Namespace (`awsvpc`).
    - Giao tiếp giữa Nextcloud và LiteLLM diễn ra trên card mạng loopback nội bộ của container, **hoàn toàn không mở port 4000 ra ngoài Internet hay mạng VPC**. Kẻ tấn công bên ngoài không có cách nào gửi request trực tiếp tới port 4000.
  - **Cơ chế xác thực không dùng khóa tĩnh (Zero Static Credentials):**
    - Nextcloud gửi API request dạng chuẩn OpenAI tới `http://127.0.0.1:4000/v1/chat/completions`.
    - LiteLLM nhận request, dịch sang AWS Bedrock format và **tự động dùng AWS ECS Task IAM Role** (thông qua link metadata `169.254.170.2`) để ký chữ ký điện tử **AWS SigV4** gọi tới Amazon Bedrock.
    - Không có bất kỳ `AWS_SECRET_ACCESS_KEY` hay OpenAI API Key nào bị lưu cứng trong mã nguồn.
- **Trích dẫn nguồn:**
  - Tài liệu cấu hình AI: `docs/docs/ai/bedrock-integration.mdx` & `litellm-bridge.mdx`.
  - Sơ đồ RAG: `presentation_assets/nextcloud_ai_rag_pipeline.png`.

Câu 24 Cơ bản – Làm thế nào để đảm bảo tính riêng tư dữ liệu (Zero Data Exfiltration) khi doanh nghiệp dùng AI của Storix?

- **Mục đích của Giám khảo:** Kiểm tra tính tuân thủ bảo mật dữ liệu doanh nghiệp khi tích hợp LLM.
- **Câu trả lời xuất sắc:** Storix đảm bảo chính sách **Zero Data Exfiltration** qua 3 cam kết kiến trúc:

1. **Chính sách bảo vệ dữ liệu của AWS Bedrock:** Khác với việc gọi OpenAI API công cộng, Amazon Bedrock cam kết theo điều khoản dịch vụ đám mây doanh nghiệp: Toàn bộ dữ liệu prompt, câu hỏi và nội dung văn bản của khách hàng **tuyệt đối KHÔNG bao giờ được AWS sử dụng để tái huấn luyện (re-train) các mô hình nền tảng (Foundation Models)**.
2. **Khu biệt phạm vi dữ liệu (Scope Locking):** Trợ lý AI chỉ đọc và tổng hợp thông tin từ chính các tệp tin mà người dùng đó có quyền truy cập trong thư mục S3 của tenant đó, không bao giờ có khả năng đọc chéo dữ liệu của tenant khác.
3. **Mã hóa đường truyền:** Toàn bộ dữ liệu trao đổi giữa LiteLLM sidecar và Amazon Bedrock endpoint được mã hóa bắt buộc qua giao thức **TLS 1.3**.

- **Trích dẫn nguồn:**
  - Slide 06: `generate_deck.py` (Lines 296–340: *Zero-Exfiltration Enterprise AI Assistant*).
  - Tài liệu Bedrock: `docs/docs/ai/bedrock-integration.mdx`.

Câu 25 Chuyên sâu – Tại sao phải patch mã nguồn `fulltextsearch_elasticsearch` trong Dockerfile khi kết nối Amazon OpenSearch?

- **Mục đích của Giám khảo:** Bắt lỗi kỹ thuật cực sâu về tính tương thích giữa Elasticsearch và OpenSearch mã nguồn mở.
- **Câu trả lời xuất sắc:**
  - **Nguyên nhân:** Bộ ứng dụng FullTextSearch của Nextcloud sử dụng thư viện PHP chính hãng của Elastic.co (`elasticsearch-php`). Từ phiên bản 7.14 trở đi, công ty Elastic.co đã cài cắm một đoạn mã kiểm tra bản quyền mang tên `ProductCheckException`:
    - Khi client gửi request đầu tiên tới máy chủ tìm kiếm, nó kiểm tra header và chuỗi phiên bản. Nếu phát hiện server là **Amazon OpenSearch Service** (chứ không phải Elasticsearch có trả phí của Elastic.co), client sẽ cố tình ném ra ngoại lệ `throw new ProductCheckException` và chặn đứng mọi hoạt động tìm kiếm.
    - Ngoài ra, trường `max_analyzed_offset` và hậu tố `.keyword` có sự khác biệt giữa hai engine.
  - **Giải pháp Patch trong Stage 1 Dockerfile của Storix:**
    - Sử dụng lệnh `sed` để vô hiệu hóa hàm kiểm tra bản quyền:

      ```
      sed -i 's/throw new ProductCheckException/return; \/\/ throw new ProductCheckException/g' \
          /apps/fulltextsearch_elasticsearch/lib/Vendor/Elastic/Elasticsearch/Traits/ProductCheckTrait.php
      ```

    - Chuẩn hóa header tương thích và loại bỏ các trường xung đột mapping qua một script Python tự động.
    - Nhờ vậy, Storix tận dụng được cụm tìm kiếm tốc độ cao **Amazon OpenSearch Service** ổn định mà không bị dính bản quyền thương mại của bên thứ ba.
- **Trích dẫn nguồn:**
  - Mã nguồn Dockerfile: `runtime/Dockerfile` (Lines 43–48).
  - Entrypoint Hook: `runtime/hooks/02-setup-opensearch.sh` (Lines 39–52).

CHỦ ĐỀ 8: BẢO MẬT, SINH TRẮC HỌC & PHÂN QUYỀN

Câu 26 Nâng cao – Cơ chế WebAuthn / Passkeys trên SuperAdmin Console bảo vệ chống tấn công Phishing như thế nào?

- **Mục đích của Giám khảo:** Đánh giá hiểu biết về công nghệ xác thực mạnh hiện đại (Strong Customer Authentication).
- **Câu trả lời xuất sắc:**
  - **Cơ chế hoạt động:**
    1. Khi SuperAdmin đăng nhập qua `https://admin.png261.dev`, trình duyệt gọi API `navigator.credentials.get()` chuẩn **FIDO2 / WebAuthn**.
    2. Người dùng chạm vân tay (Touch ID), nhận diện khuôn mặt (Face ID) hoặc cắm khóa phần cứng YubiKey.
    3. Thiết bị phần cứng an toàn (Secure Enclave) trên máy tính tạo ra một chữ ký số bằng **Khóa Riêng (Private Key)** nằm bất biến trong chip bảo mật và gửi chữ ký kèm Challenge lên server.
    4. Server sử dụng **Khóa Công Khai (Public Key)** đã lưu trong DynamoDB `admin-users` để kiểm tra chữ ký.
  - **Chống Phishing tuyệt đối:** Khóa riêng WebAuthn gắn chặt (scoped) với tên miền `admin.png261.dev`. Nếu kẻ tấn công tạo một trang web giả mạo (ví dụ `admin-hack.png261.dev`), trình duyệt sẽ nhận diện sai Origin và **từ chối giải phóng chữ ký số**, giúp hệ thống miễn nhiễm 100% với các cuộc tấn công lừa đảo đánh cắp mật khẩu.
- **Trích dẫn nguồn:**
  - File cấu hình DB: `docs/docs/control-plane/dynamodb-registry.mdx` (Mục 2: *Admin Users Table WebAuthn Credentials*).
  - Mã nguồn Console: `console/src/authProvider.ts` & `console/src/pages/login/index.tsx`.

Câu 27 Trung bình – Cơ chế WebDAV Exclusive Lock (HTTP 423) và Bitmask Permissions hoạt động ra sao?

- **Mục đích của Giám khảo:** Kiểm tra kiến thức về giao thức WebDAV RFC 4918 và cơ chế phân quyền tệp tin.
- **Câu trả lời xuất sắc:**
  - **WebDAV Exclusive Lock (RFC 4918):**
    - Khi một thành viên mở chỉnh sửa tệp tin tài liệu (Word/Excel) qua WebDAV Client, client gửi phương thức `LOCK` lên máy chủ. Nextcloud cấp phát một mã khóa `Lock-Token` có thời hạn (timeout).
    - Bất kỳ người dùng nào khác cố tình gửi request `PUT` hoặc `DELETE` lên tệp tin này khi chưa có token sẽ nhận ngay mã lỗi `HTTP 423 Locked`. Điều này ngăn chặn hiện tượng mất mát dữ liệu do người sau ghi đè lên công sức của người trước.
  - **Mặt nạ bit phân quyền (Bitmask Permissions):**
    - Quyền trên từng tệp/thư mục được lưu dưới dạng số nguyên bitmask nhị phân từ 1 đến 16:
      - `1`: Quyền Đọc (Read / Share permission)
      - `2`: Quyền Cập nhật (Update)
      - `4`: Quyền Tạo mới (Create)
      - `8`: Quyền Xóa (Delete)
      - `16`: Quyền Chia sẻ lại (Share)

    - Một liên kết chia sẻ chỉ cho phép đọc sẽ có giá trị quyền là `1`. Thư mục cho phép thêm file nhưng cấm xóa có quyền là `1 + 2 + 4 = 7`.
- **Trích dẫn nguồn:**
    - Slide 03 & 05: `generate_deck.py` (Lines 151–155 & Lines 259–264).
    - Tiêu chuẩn quốc tế: IETF RFC 4918 (HTTP Extensions for Web Distributed Authoring and Versioning).

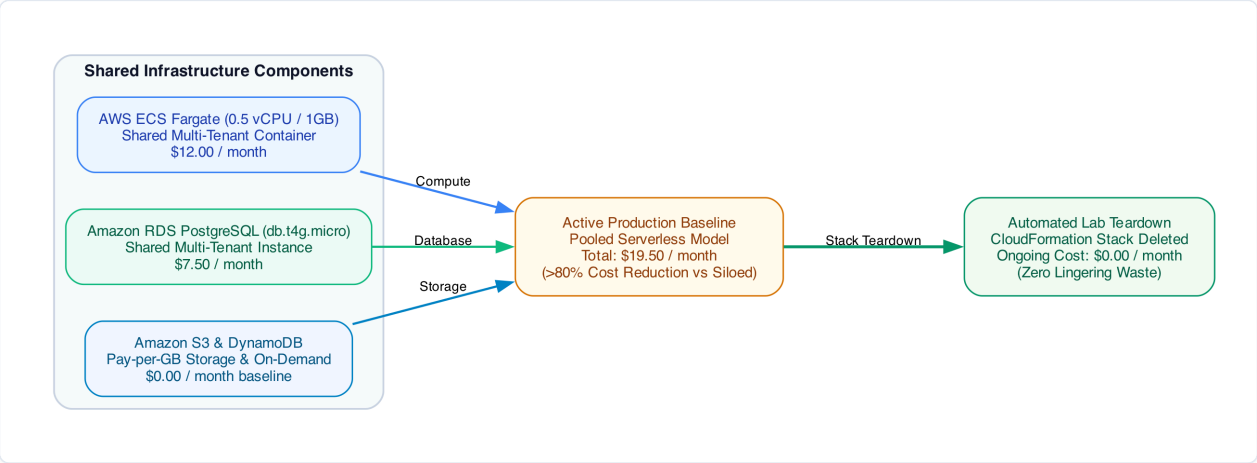
Câu 28 Cơ bản – Tính năng File Drop Inbox (Hòm thư nhận tài liệu ẩn danh) được hiện thực như thế nào?

- **Mục đích của Giám khảo:** Đánh giá tính ứng dụng thực tế trong kịch bản doanh nghiệp thu thập hồ sơ thầu, CV ứng viên.
- **Câu trả lời xuất sắc:**
    - **Yêu cầu nghiệp vụ:** Doanh nghiệp cần khách hàng hoặc đối tác bên ngoài nộp tài liệu vào thư mục công ty, nhưng đối tác **tuyệt đối không được nhìn thấy các tài liệu do người khác đã nộp trước đó**.
    - **Hiện thực trong Storix:**
        - Tenant Member tạo một Public Link Share cho một thư mục và chọn chế độ **File Drop (Upload only)**.
        - Bitmask quyền hạn của link được set chính xác bằng `4` (chỉ có bit `CREATE`, không có bit `READ` hay `UPDATE`).
        - Khi đối tác bên ngoài truy cập link qua trình duyệt, giao diện chỉ hiển thị duy nhất một vùng kéo thả tệp tin (Drag & Drop zone). Khi tải tệp lên xong, file được lưu thẳng vào S3 Vault Bucket và người tải lên không thấy danh sách file trong thư mục.
- **Trích dẫn nguồn:**
    - Sơ đồ cộng tác: `presentation_assets/external_collaboration_flow.png`.
    - Báo cáo use-case: `reports/2026-10-05/diagrams/diagram2_usecase.drawio`.

CHỦ ĐỀ 9: FINOPS & KINH TẾ HỌC ĐIỆN TOÁN (SCALE-TO-ZERO)

> **Câu 29** Nâng cao **– Khi kích hoạt FinOps Suspend một Tenant, những tài nguyên AWS nào thay đổi và tại sao đạt được chi phí $0.00 compute?**

- **Mục đích của Giám khảo:** Đây là điểm nhấn FinOps quan trọng nhất của Storix, giám khảo sẽ kiểm tra xem chi phí có thực sự về $0 hay không.
- **Sơ đồ kiến trúc minh họa:**



  Sơ đồ Draw.io gốc: diagram4_pillars_architecture.drawio

- **Câu trả lời xuất sắc:**
  - Khi kích hoạt **SUSPEND** (thông qua SuperAdmin Console hoặc API):
    1. **Tầng Compute:** Lệnh `UpdateService` trên ECS Fargate cập nhật `desiredCount = 0`. Toàn bộ container Fargate của tenant lập tức dừng hoạt động. Vì Fargate tính tiền theo giây dựa trên số lượng task đang chạy, khi số task = 0 thì **chi phí Compute rơi chính xác về $0.00/giờ**.
    2. **Tầng Load Balancer:** Rule trên ALB được cập nhật để chuyển hướng lưu lượng truy cập sang một trang thông báo bảo trì hoặc trả về mã lỗi HTTP 503, không gửi request vào cụm tính toán.
    3. **Tầng Database:** RDS Proxy nhận thấy container đã ngắt kết nối, lập tức đóng các pooled connection, giải phóng RAM và connection slot cho các tenant đang active khác.
    4. **Tầng Storage (Bảo toàn dữ liệu):** Dữ liệu trên EFS và S3 được giữ nguyên vẹn 100%. Các tệp tin cấu hình EFS nhàn rỗi sau 30 ngày tự động chuyển sang **EFS Infrequent Access (IA)** giúp giảm 92% giá lưu trữ. Dữ liệu S3 chuyển sang Glacier Instant Retrieval giúp giảm 80% chi phí.
  - Khi nhấn **RESUME**: Step Functions đưa `desiredCount = 1`, Fargate task khởi động lại nhanh chóng nhờ AWS SOCI Indexing, kết nối lại database và sẵn sàng phục vụ chỉ sau **15 - 20 giây**.
- **Trích dẫn nguồn:**
  - Báo cáo thực nghiệm: `reports/2026-10-05/report.md` (Section 4.3: *FinOps Lifecycle Scaling Suspend & Resume*).
  - Tài liệu chi tiết: `docs/docs/tenants/suspension-resumption.mdx`.

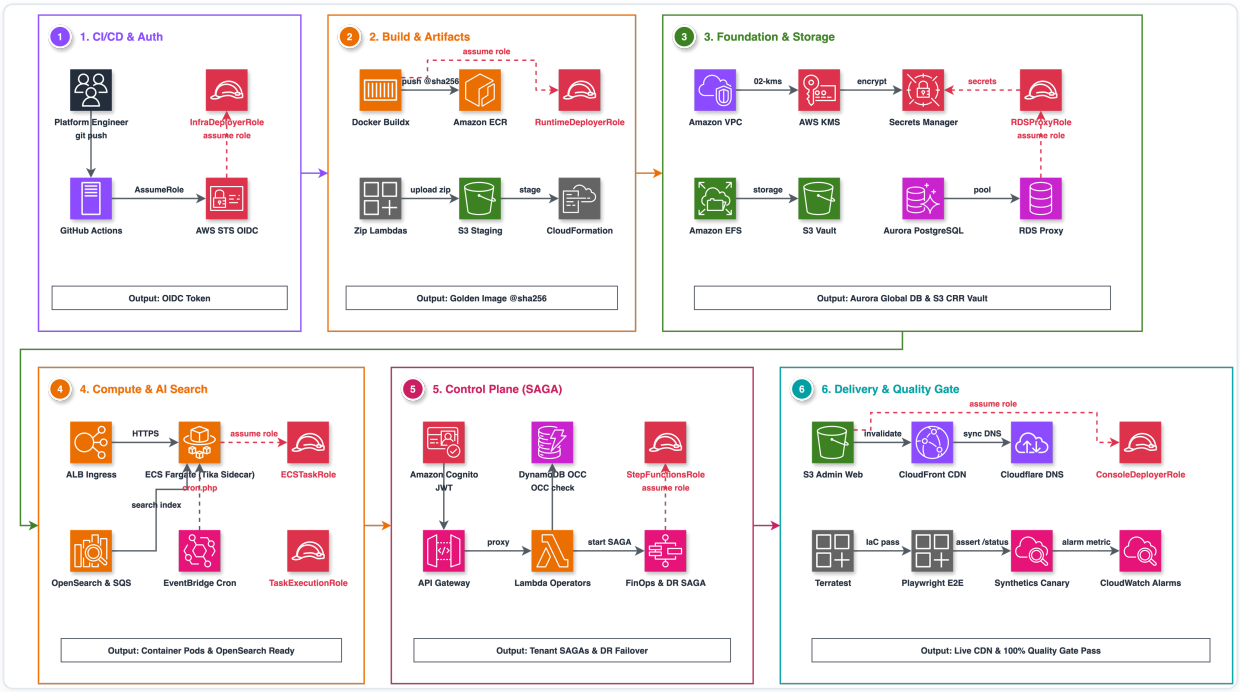
Câu 30 Trung bình – Phân tích so sánh Tổng chi phí sở hữu (TCO) giữa Storix, Mô hình máy chủ ảo truyền thống và Public SaaS?

- **Mục đích của Giám khảo:** Đánh giá tính khả thi thương mại và bài toán đầu tư ROI.
- **Câu trả lời xuất sắc:** Giả sử một doanh nghiệp có **10 phòng ban / công ty con (10 tenants)** với tổng cộng **200 nhân viên**:
  - **Phương án 1 - Public SaaS (Google Workspace Business Plus - $18/người/tháng):**
    - Chi phí hàng tháng: 200 × $18 = **$3, 600/**tháng ($43,200/năm). Chi phí tăng tuyến tính khi tuyển thêm nhân sự.
  - **Phương án 2 - 10 Máy chủ EC2 riêng lẻ (t4g.small ~ $15/tháng + EBS + Snapshot):**
    - Chi phí hàng tháng: 10 × $25 = **$250/**tháng, nhưng máy chủ chạy 24/7 lãng phí vào ban đêm, không có tính năng co giãn tự động và tốn nhân sự bảo trì, vá lỗi hệ điều hành.
  - **Phương án 3 - Nền tảng Storix trên AWS:**
    - Ban ngày: 10 Fargate tasks Graviton chạy 10 tiếng/ngày ≈ $27/tháng.
    - Ban đêm: Tự động Suspend 8 tenant nhàn rỗi về 0 task → Compute = $0.
    - S3 Vault Storage (2TB) qua Intelligent-Tiering ≈ $30/tháng.
    - RDS PostgreSQL 16 Multi-AZ chia sẻ qua RDS Proxy ≈ $60/tháng.
    - Tổng chi phí: **~$120 - $150/tháng** (chỉ bằng **4%** chi phí của Google Workspace), nhưng doanh nghiệp sở hữu 100% dữ liệu và tích hợp GenAI bảo mật.
- **Trích dẫn nguồn:**
  - Slide 02 & Slide 07: `generate_deck.py` (Lines 86–114 & Lines 373–384).

CHỦ ĐỀ 10: CI/CD, INFRASTRUCTURE AS CODE & PROCESS DESTROY

Câu 31 Nâng cao **– Hãy trình bày cấu trúc 7 Stacks CloudFormation và cách quản lý phụ thuộc (Cross-Stack References)?**

- **Mục đích của Giám khảo:** Kiểm tra kỹ năng kiến trúc hạ tầng dạng mã (IaC) chuẩn doanh nghiệp.
- **Sơ đồ kiến trúc minh họa:**



Sơ đồ Draw.io gốc: storix_deployment_pipeline.drawio

- **Câu trả lời xuất sắc:** Hạ tầng Storix được chia tách thành **7 Modules CloudFormation** độc lập theo thứ tự phân tầng phụ thuộc:
  1. `01-networking` : Tạo VPC `10.0.0.0/16` , Dual-AZ Subnets, Internet Gateway và S3 Gateway Endpoint. Export: `VPCId` , `SubnetIds` .
  2. `02-security` : Tạo KMS Customer Managed Keys (CMK), Security Groups (ALB, ECS, RDS, EFS) và IAM Execution Roles.
  3. `03-storage` : Tạo Amazon EFS FileSystem và S3 Multi-Tenant Vault Bucket kèm chính sách Intelligent-Tiering.
  4. `04-database` : Tạo Amazon RDS PostgreSQL 16 Multi-AZ, Secrets Manager credentials và Amazon RDS Proxy.
  5. `05-compute` : Tạo cụm ECS Cluster, Application Load Balancer, Wildcard HTTPS Listener và EventBridge Cron Runner.
  6. `06-superadmin` : Tạo API Gateway Ingress, Step Functions SAGA State Machine, DynamoDB Registry, và Control Plane Lambda.
  7. `07-observability` : Tạo CloudWatch Synthetics Canary, CloudWatch Telemetry Dashboard và Alarms.
  - **Quản lý phụ thuộc:** Các stack tầng trên sử dụng hàm `Fn::ImportValue` để đọc các giá trị Output được Export từ các stack tầng dưới, đảm bảo tính phân tách module và tính bất biến (Immutability).

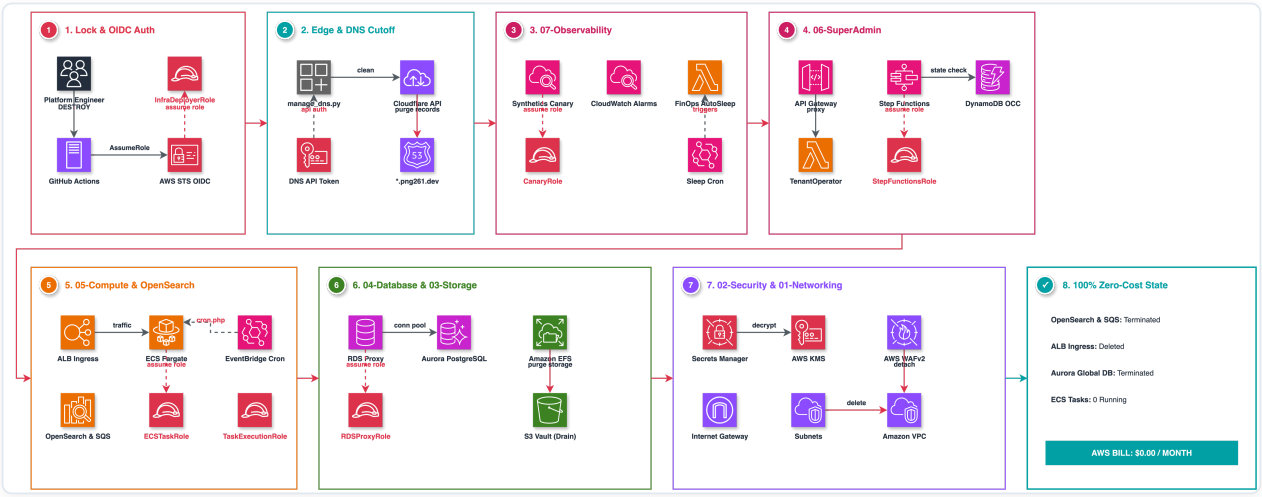
- **Trích dẫn nguồn:**
  - Thư mục IaC: `infra/infra/modules/` (Từ `01-networking` đến `07-observability`).
  - Danh mục Stacks thực tế: `reports/2026-10-05/report.md` (Bảng Section 2, Lines 51–64).

Câu 32 Chuyên sâu – Tại sao Storix loại bỏ hoàn toàn Long-Lived AWS Keys trong GitHub Actions và thay bằng OIDC STS?

- **Mục đích của Giám khảo:** Kiểm tra tiêu chuẩn an ninh hiện đại trong bảo mật DevSecOps CI/CD.
- **Câu trả lời xuất sắc:**
  - **Nguy cơ của Long-Lived Keys:** Khi lưu `AWS_ACCESS_KEY_ID` và `AWS_SECRET_ACCESS_KEY` trong GitHub Secrets, nếu repo bị lộ hoặc bị tấn công qua supply chain, kẻ xấu có thể đánh cắp khóa này để chiếm toàn quyền tài khoản AWS vĩnh viễn mà không có thời hạn hết hạn.
  - **Giải pháp OpenID Connect (OIDC) qua AWS STS của Storix:**
    1. AWS IAM cấu hình một **OIDC Identity Provider** tin cậy `token.actions.githubusercontent.com`.
    2. Một IAM Role mang tên `InfraDeployerRole` được tạo với chính sách `AssumeRoleWithWebIdentity`, chỉ cho phép repo `FilestoreCloud/infra` trên nhánh `main` được quyền assume.
    3. Khi GitHub Actions chạy: GitHub sinh ra một JSON Web Token (JWT) ngắn hạn được ký mật mã.
    4. GitHub Actions gửi JWT này tới **AWS STS (Security Token Service)** để xin cấp phát credentials tạm thời (thời gian sống chỉ từ **15 đến 60 phút**).
    5. Không có bất kỳ access key tĩnh nào được lưu trữ, triệt tiêu 100% nguy cơ rò rỉ khóa bảo mật.
- **Trích dẫn nguồn:**
  - Sơ đồ Block 1 CI/CD Auth: `docs/images/storix_block1_cicd_auth.png`.
  - Tài liệu IAM Roles: `docs/deployment_pipeline_architecture.md` (Mục: *Bản Đồ AWS IAM Roles Trong Toàn Bộ Hệ Thống*).

Câu 33 Nâng cao – Trình bày quy trình 8-Stage Process Destroy và các biện pháp bảo vệ chống xóa nhầm?

- **Mục đích của Giám khảo:** Đánh giá tính kỷ luật trong quản lý vòng đời tài nguyên và thu hồi hạ tầng không tốn phí.
- **Sơ đồ kiến trúc minh họa:**



Sơ đồ Draw.io gốc: storix_process_destroy.drawio

- **Câu trả lời xuất sắc:**
  - **Lớp bảo vệ Safeguard:** Workflow hủy hạ tầng (`destroy.yml`) bắt buộc người vận hành phải nhập chính xác biến môi trường `confirm_destruction == 'DESTROY'` . Bất kỳ giá trị nào khác đều lập tức hủy lệnh.
  - **Quy trình thu hồi 8 giai đoạn ngược chiều:**
    - *Giai đoạn 1:* Xác thực OIDC STS ngắn hạn qua role `InfraDeployerRole` .
    - *Giai đoạn 2:* Gọi API DNS (Cloudflare/Route 53) xóa toàn bộ bản ghi `*.png261.dev` , ngắt ngay lập tức lưu lượng ngoài Internet.
    - *Giai đoạn 3:* Hủy Stack `07-Observability` (tắt Synthetics Canary, tránh Canary báo động giả khi các tầng sau bị xóa).
    - *Giai đoạn 4:* Hủy Stack `06-SuperAdmin` (tắt Control Plane API, Step Functions, DynamoDB).
    - *Giai đoạn 5:* Hủy Stack `05-Compute` (dừng toàn bộ Fargate tasks, xóa ECS Services và xóa ALB).
    - *Giai đoạn 6:* Hủy Stack `04-Database` (thu hồi RDS Proxy và xóa PostgreSQL cluster).
    - *Giai đoạn 7:* Hủy Stack `03-Storage` (chạy script dọn sạch các version trong S3 Vault Bucket và xóa EFS FileSystem).
    - *Giai đoạn 8:* Hủy Stack `02-Security` và `01-Networking` (xóa KMS CMK, Security Groups, Subnets và VPC).
  - Đưa tài khoản AWS về trạng thái **Zero-Cost sạch 100%**.
- **Trích dẫn nguồn:**
  - File tài liệu: `docs/deployment_pipeline_architecture.md` (Mục: *Quy Trình Hủy Diệt & Thu Hồi Hạ Tầng - Process Destroy*).
  - File vẽ: `docs/storix_process_destroy.drawio` .

CHỦ ĐỀ 11: ĐO LƯỜNG ĐỘ TIN CẬY & SLA 24/7

Câu 34 Trung bình – Cơ chế giám sát SLA 24/7 bằng AWS CloudWatch Synthetics Canary hoạt động như thế nào?

- **Mục đích của Giám khảo:** Kiểm tra xem hệ thống đo lường độ tin cậy bằng cách nào (Synthetic Monitoring vs Passive Monitoring).
- **Câu trả lời xuất sắc:**
  - Thay vì chỉ đợi người dùng gặp lỗi mới phát hiện, Storix triển khai **CloudWatch Synthetics Canary**:
    - Một kịch bản Node.js chạy trên framework **Playwright Headless Browser** được CloudWatch thực thi tự động **định kỳ 5 phút một lần suốt 24/7**.
    - Canary gửi request HTTPS thực tế tới endpoint kiểm tra sức khỏe của từng tenant: `https://<tenant_id>.png261.dev/status.php` và trang SuperAdmin Console.
    - Canary kiểm tra 3 điều kiện:
      1. Mã phản hồi HTTP phải là `200 OK`.
      2. Thời gian phản hồi tổng thể (Latency) phải `< 1,500ms`.
      3. Nội dung phản hồi JSON phải chứa chuỗi `"installed": true` và `"maintenance": false`.
    - Canary chụp ảnh màn hình lưu vào S3 làm bằng chứng nghiệm thu SLA. Nếu có 2 lần kiểm tra liên tiếp thất bại, CloudWatch Alarm lập tức bắn cảnh báo sự cố cho đội ngũ vận hành.
- **Trích dẫn nguồn:**
  - File CloudFormation Observability: `infra/infra/modules/07-observability/template.yaml` (Resource: `SyntheticsCanary`).
  - Tài liệu kiến trúc: `docs/docs/architecture-overview.mdx` (Mục 1, Dòng 29: *24/7 SLA Probing via CloudWatch Synthetics Canary*).

Câu 35 Nâng cao – Trình bày kiến trúc Bảng ghi nhật ký kiểm toán bất biến (Immutable Audit Ledger) trên DynamoDB?

- **Mục đích của Giám khảo:** Kiểm tra khả năng đáp ứng các tiêu chuẩn tuân thủ bảo mật doanh nghiệp (Compliance: SOC 2, ISO 27001).
- **Câu trả lời xuất sắc:**
  - Mọi hành động nhạy cảm trên SuperAdmin Console (Đăng nhập, Provision, Suspend, Resume, Nâng cấp Image, chạy lệnh OCC) đều được ghi vào bảng DynamoDB `prod-nextcloud-audit-logs`.
  - **Cấu trúc bản ghi kiểm toán:**

```
{
  "event_id": "37f73442-1077-4871-94f2-98483af4e2bb",
  "timestamp": "2026-10-18T16:39:31.390193+00:00",
  "actor": "admin",
  "role": "SuperAdmin",
  "action": "SUSPEND_TENANT",
  "target": "tenant/marvel",
  "status": "SUCCESS",
  "source_ip": "115.79.x.x",
  "user_agent": "Mozilla/5.0 ...",
  "ttl": 1823877571
}
```

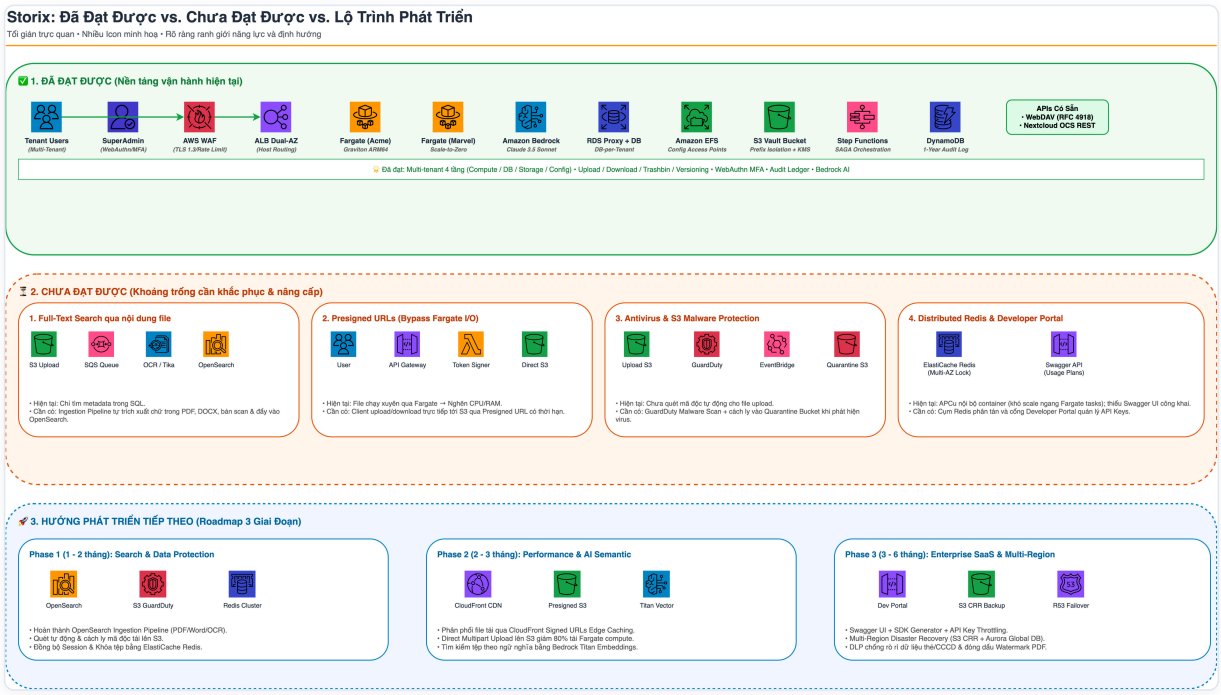
- **Tính bất biến (Immutability):** IAM Role của Console Lambda chỉ được cấp quyền `dynamodb:PutItem`, **tuyệt đối không có quyền** `UpdateItem` **hay** `DeleteItem` trên bảng Audit Logs.
- **Tự động lưu trữ và thanh lý:** Thiết lập trường `ttl` (Time-To-Live) tự động xóa sau 1 năm hoặc đồng bộ sang S3 Glacier để lưu trữ kiểm toán lâu dài với chi phí cực thấp.

- **Trích dẫn nguồn:**
  - Tài liệu DynamoDB: `docs/docs/control-plane/dynamodb-registry.mdx` (Mục 3: *Audit Logs Ledger*).

CHỦ ĐỀ 12: BỘ CÂU HỎI BẪY & HẠN CHẾ KIẾN TRÚC

> **Câu 36** ⚠ CÂU HỎI BẪY **– "Nếu số lượng khách hàng tăng lên 1,000 tenants, hệ thống có bị nghẽn ở ALB hoặc RDS không? Giới hạn kỹ thuật là gì và giải pháp ra sao?"**

- **Bẫy của Giám khảo:** Xem sinh viên có biết giới hạn vật lý của kiến trúc mình thiết kế hay chỉ nói "hệ thống em scale vô tận".
- **Sơ đồ kiến trúc minh họa (Roadmap tiến hóa):**



Sơ đồ Draw.io gốc: storix_gap_roadmap_minimalist.drawio

- **Câu trả lời xuất sắc:**
  - "Thưa Hội đồng, đây là một câu hỏi rất sâu về các ngưỡng giới hạn vật lý (Physical Quotas):
    1. **Nghẽn tại ALB Listener Rules:** Một Application Load Balancer của AWS mặc định chỉ hỗ trợ tối đa **100 Rules**. Khi số lượng tenant vượt quá 100, ta không thể tạo mỗi tenant một rule host-header riêng lẻ trên 1 ALB.
       - *Giải pháp:* Chuyển sang mô hình **ALB Sharding** (mỗi ALB phụ trách 50–80 tenants), hoặc sử dụng **Wildcard Host-Header Rule (`*.png261.dev`)** kết hợp với một tầng Ingress Controller nhẹ (Envoy Proxy / AWS Cloud Map Service Discovery) để định tuyến động tới đúng container Target Group mà không bị giới hạn số rule của ALB.
    2. **Nghẽn tại RDS PostgreSQL Single Cluster:** Một instance PostgreSQL đơn lẻ dù cấu hình lớn đến đâu cũng bị giới hạn về I/O disk và số lượng connection tối đa.
       - *Giải pháp (Lộ trình Phase 1):* Di chuyển sang **Amazon Aurora Serverless v2 Multi-Cluster** kết hợp Sharding theo cụm Tenant (ví dụ Cluster 1 phục vụ tenant 1–200, Cluster 2 phục vụ 201–400)."
- **Trích dẫn nguồn:**
  - Slide 10: `generate_deck.py` (Lines 517–582: *Limitations & Evolutionary Roadmap*).
  - Tài liệu Roadmap: `docs/images/storix_gap_roadmap_minimalist.png`.

Câu 37 ⚠ CÂU HỎI BẪY – "Tại sao trong mã nguồn `runtime/storix-entrypoint.sh` lại thấy lệnh xóa bỏ Redis? Không có Redis thì Nextcloud có bị chết khóa giao dịch (Transaction Locking) không?"

- **Bẫy của Giám khảo:** Kiểm tra xem sinh viên có tự viết mã nguồn hay không, và có hiểu cơ chế Locking của Nextcloud.
- **Câu trả lời xuất sắc:**
  - "Dạ thưa Thầy/Cô, trong các hướng dẫn Nextcloud phổ biến, Redis thường được dùng cho 2 nhiệm vụ: Lưu PHP Session và khóa tệp tin phân tán (`memcache.locking`).
  - Tuy nhiên, trong thiết kế **Dedicated Container-Per-Tenant kết hợp Zero-NAT** của Storix:
    1. Nếu mỗi tenant nhỏ dựng một cụm Amazon ElastiCache Redis riêng thì chi phí tối thiểu tăng thêm **$15 - $20/tháng/tenant**, phá vỡ mục tiêu FinOps tối ưu chi phí.
    2. Storix chuyển tầng Session sang quản lý trực tiếp bằng cơ chế mã hóa cookie an toàn của Nextcloud kết hợp session table trong cơ sở dữ liệu.
    3. Tầng khóa tệp tin (File Locking) được Nextcloud hỗ trợ native chuyển sang sử dụng khóa giao dịch ACID của **PostgreSQL 16 thông qua bảng** `oc_file_locks` .
    4. Trong `storix-entrypoint.sh` , em chủ động unset `REDIS_HOST` và xóa `redis-session.ini` nếu không có biến môi trường Redis, nhằm ngăn việc container mất thời gian cố kết nối tới localhost port 6379 gây treo tiến trình khởi động. Đây là chủ đích tối ưu có tính toán trước của nhóm!"
- **Trích dẫn nguồn:**
  - Mã nguồn container: `runtime/storix-entrypoint.sh` (Lines 4–9: *Unset REDIS_HOST and remove redis-session.ini*).

Câu 38 ⚠ CÂU HỎI BẪY – "Tại sao lại chọn AWS ECS Fargate mà không dùng Kubernetes (Amazon EKS) cho hoành tráng?"

- **Bẫy của Giám khảo:** Xem sinh viên có mắc bệnh "thích công nghệ phức tạp quá mức cần thiết" (Over-engineering) hay không.
- **Câu trả lời xuất sắc:**
  - "Thưa Hội đồng, đây là quyết định kiến trúc được cân nhắc rất kỹ dựa trên nguyên tắc **Right Tool for the Right Job**:
    1. **Chi phí cụm nền tảng (Cluster Management Cost):** Amazon EKS tính phí cố định **$72/tháng chỉ riêng tiền Control Plane**, chưa kể chi phí chạy tối thiểu 2 worker nodes EC2 để duy trì CoreDNS và Kube-proxy. Trong khi đó, **ECS Control Plane hoàn toàn miễn phí ($0/tháng)**.
    2. **Khả năng Scale-to-Zero thực thụ:** Trên EKS, rất khó để giảm chi phí compute về $0.00 vì cụm node Kubernetes vẫn phải chạy nền. Trên ECS Fargate, khi tenant bị Suspend về 0 task, ta không tốn một xu nào cho compute.
    3. **Đơn giản hóa vận hành (Operational Simplicity):** EKS đòi hỏi bảo trì etcd, nâng cấp phiên bản Kubernetes định kỳ 3 tháng/lần, quản lý CNI plugin. ECS Fargate là Serverless Container, AWS tự quản lý hạ tầng bên dưới, giúp nhóm tập trung 100% vào nghiệp vụ SaaS và bảo mật ứng dụng."
- **Trích dẫn nguồn:**
  - Tài liệu kiến trúc: `docs/docs/architecture-overview.mdx` (Mục 1: *Key Architectural Layers*).

Câu 39 Nâng cao – Hạn chế lớn nhất hiện tại của Storix là gì và Lộ trình 4 giai đoạn tiến hóa (Roadmap Phases 1–4) như thế nào?

- **Mục đích của Giám khảo:** Đánh giá tính trung thực, khả năng tự nhìn nhận khuyết điểm và tầm nhìn phát triển sản phẩm.

- **Câu trả lời xuất sắc:**
  - **Hai hạn chế lớn nhất hiện tại:**
    1. *Độ trễ Cold Start khi Resume:* Khi resume từ 0 task, dù có AWS SOCI thì thời gian Fargate tải container và khởi động Apache vẫn mất khoảng 15–20 giây.
    2. *Chia sẻ trực tiếp liên-tenant (Cross-Tenant Collaboration):* Do áp dụng ranh giới cô lập tuyệt đối, hai công ty trên hai tenant khác nhau hiện chỉ có thể chia sẻ tệp qua Public URL có mật khẩu, chưa thể mount chéo thư mục làm việc chung một cách tự nhiên.
  - **Lộ trình 4 giai đoạn tiến hóa (Strategic Roadmap):**
    - **Phase 1 (Database Serverless):** Chuyển sang Aurora Serverless v2 + extension `pgvector` để tối ưu chi phí database theo giây và lưu trữ embedding vector trực tiếp.
    - **Phase 2 (Lambda MicroVMs):** Ứng dụng **AWS Lambda MicroVMs (Firecracker Snapshot Resumable Compute)** để đóng băng và khôi phục bộ nhớ RAM container, cắt giảm thời gian khởi động từ 15s xuống **dưới 3 giây**.
    - **Phase 3 (OpenSearch Serverless):** Xây dựng cụm vector chuyên biệt phục vụ tìm kiếm ngữ nghĩa (Semantic Search) trên hàng triệu tài liệu.
    - **Phase 4 (AI Workflow Automation):** Tự động hóa pipeline tổng hợp tài liệu qua EventBridge: Ngay khi user upload tài liệu PDF/Word lên S3, hệ thống tự kích hoạt Bedrock tóm tắt và sinh thẻ tag tự động.
- **Trích dẫn nguồn:**
  - Slide 10: `generate_deck.py` (Lines 533–580: *Trade-Offs & Strategic Roadmap Phases 1–4*).
  - Sơ đồ Roadmap: `docs/images/storix_gap_roadmap_minimalist.png`.

Câu 40 Chuyên sâu – Nếu muốn thiết kế khôi phục sau thảm họa (Disaster Recovery - Multi-Region Active/Passive), Storix sẽ mở rộng như thế nào?

- **Mục đích của Giám khảo:** Đánh giá tầm nhìn kiến trúc cấp độ Enterprise Solutions Architect.
- **Câu trả lời xuất sắc:**
  - Để đạt mục tiêu **RPO (Recovery Point Objective) < 5 phút** và **RTO (Recovery Time Objective) < 15 phút**, Storix sẽ mở rộng từ Singapore (`ap-southeast-1`) sang Region dự phòng Jakarta (`ap-southeast-3`):
    1. **Tầng Lưu trữ (S3 & EFS):**
       - Kích hoạt **S3 Cross-Region Replication (CRR)** bất đồng bộ giữa Singapore Vault và Jakarta Vault Bucket.
       - Dùng **AWS Backup** sao lưu EFS Access Point config hàng ngày sang Region dự phòng.
    2. **Tầng Cơ sở dữ liệu (PostgreSQL):** Thiết lập **Amazon RDS Cross-Region Read Replica** tại Jakarta. Dữ liệu thay đổi được đồng bộ liên tục qua mạng trục AWS Backbone.
    3. **Tầng Điều phối & Đồng bộ thông điệp:** Sử dụng **Amazon SQS / EventBridge Cross-Region Replication** để đồng bộ sự kiện tenant lifecycle.
    4. **Tầng Chuyển mạch (Failover):** Sử dụng **Amazon Route 53 DNS Failover với Health Check**: Nếu toàn bộ cụm ALB tại Singapore không phản hồi sau 3 lần kiểm tra, Route 53 tự động chuyển hướng toàn bộ traffic sang ALB tại Jakarta, đồng thời kích hoạt Step Functions thúc đẩy Read Replica lên thành Master Database.
- **Trích dẫn nguồn:**
  - Script kiến trúc Multi-Region: `update_multiregion_sqs_diagrams.py`.
  - Sơ đồ Master: `docs/images/storix_system_architecture_master.png`.

🎖️ CẨM NANG 5 QUY TẮC "VÀNG" ĐỂ ĐẠT ĐIỂM XUẤT SẮC TRƯỚC HỘI ĐỒNG:

1. **Quy tắc "Cảm ơn & Phân tách 3 ý":**
   - Khi nhận câu hỏi khó: *"Em cảm ơn câu hỏi rất thực tế của Thầy/Cô. Để giải quyết vấn đề này, nhóm em đã thiết kế dựa trên 3 trụ cột chính: Thứ nhất là..., Thứ hai là..., Và thứ ba là..."*. Việc chia 3 ý giúp bạn bình tĩnh và câu trả lời nghe cực kỳ logic.
2. **Quy tắc "Nhìn hình nói chuyện" (Always Reference Architecture Diagrams):**
   - Đừng trả lời chay! Mỗi câu hỏi kiến trúc đều có sơ đồ Draw.io đi kèm. Hãy mở đúng slide hoặc ảnh zoom của block đó (`zoom_block_1` đến `zoom_block_5`) và chỉ trực tiếp vào component: *"Như Thầy/Cô có thể thấy trên sơ đồ Ingress Security..."*.
3. **Quy tắc "Số liệu thực nghiệm đè bẹp lý thuyết" (Data-Driven Defense):**
   - Luôn gắn câu trả lời với môi trường thật: *"Hệ thống đã được kiểm chứng trên AWS Singapore account 649519997247, chạy thành công trên 3 tenant acme, marvel, tesla, vượt qua 9/9 test suites với mã băm SHA-256 khớp 100%"*.
4. **Quy tắc "Không giấu dốt - Nhận diện Trade-off":**
   - Nếu bị hỏi về điểm yếu (như Cold start hay ALB 100-rule limit), **hãy dũng cảm thừa nhận và đưa ra giải pháp Roadmap**. Hội đồng đánh giá cực cao các kỹ sư biết rõ giới hạn của hệ thống mình làm ra.
5. **Quy tắc "Bộ ba Zero":**
   - Luôn nhắc lại 3 từ khóa định vị thương hiệu của đồ án: **Zero-Code-Modification** (bảo trì bền vững), **Zero-NAT** (tối ưu chi phí), và **Zero-Data-Exfiltration** (an toàn GenAI tuyệt đối).